



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

03.3: Multimedia Networking

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

Advanced Network Application Paradigms

This 26-hour module is a deep dive into the next generation of network technologies, structured into four key topics that explore how networks are being transformed to support modern, demanding applications.

Part 1: Mobile Edge Computing (MEC) (7 hours)

- This topic explores **Mobile Edge Computing (MEC)**, a distributed paradigm that brings cloud capabilities to the network's edge. We'll see how MEC drastically reduces latency by processing data locally, enabling real-time applications like **AR**, **VR**, and **video analytics** that are impossible with traditional centralized clouds.

Part 2: Network Slicing (7 hours)

- This topic explores **Network Slicing**, a key 5G technology that partitions a single physical network into multiple virtual "slices." Each slice can be customized for specific performance requirements like bandwidth and latency, serving applications from **IoT devices** to **autonomous vehicles**.

Part 3: Multimedia Networking (7 hours)

- This topic is dedicated to the challenges of delivering high-quality multimedia content. We'll cover protocols and techniques for ensuring quality of service (QoS) and managing jitter and latency. Key topics include **adaptive bitrate streaming** and the role of **CDNs**.

Part 4: Distributed Machine Learning (DML) (6 hours)

- This topic explores **Distributed Machine Learning (DML)**, where ML models are trained and run across a network of devices. It enables local computation, reducing reliance on a central server. We'll discuss **federated learning**, where a model is trained on devices without sharing raw data, and its benefits for **personalized assistants** and **IoT analytics**.

Foundations of Multimedia Networking



Multimedia Networking

In the previous parts, we explored how to create flexible infrastructure (MEC) and customized virtual networks (Slicing).

Now, we focus on one of the most demanding and dominant application types that runs on these networks: **multimedia**.

This part provides a deep dive into the unique **networking challenges**, the **protocols**, and the **distributed systems** that have been engineered to deliver high-quality video and audio experiences over best-effort IP networks.

What Makes Multimedia Networking Unique?

Multimedia traffic is fundamentally different from traditional data traffic (like web browsing or email). It is highly sensitive to network performance variations.

Key Challenges:

- **High Bandwidth Requirement:** Uncompressed video requires an enormous amount of bandwidth. Even with compression, 4K video streams can consume 15-25 Mbps, orders of magnitude more than most data applications.
- **Latency Sensitivity:** For interactive, real-time applications (VoIP, video conferencing), end-to-end delay must be low (typically < 150-200 ms) to allow for natural conversation.
- **Jitter Sensitivity: Jitter** is the *variation* in packet arrival times. High jitter is disastrous for multimedia because it causes the playback to be choppy and distorted. Packets must arrive at a constant, predictable rate.
- **Loss Tolerance (with limits):** Unlike a file transfer where a single lost bit can corrupt the entire file, multimedia streams can often tolerate some packet loss, as codecs are designed with error resilience. However, excessive loss severely degrades quality.

The Network's View of a Multimedia Stream

From a network engineering perspective, multimedia flows have distinct characteristics that differentiate them from bursty data traffic (like web browsing).

Key Network Characteristics:

- **Long-Lived Flows:** A video stream can last for minutes or hours, creating a persistent, high-volume flow between a client and server.
- **High Volume:** Even with compression, these flows consume a significant, sustained amount of bandwidth (Mbps to tens of Mbps).
- **Predictability (in bursts):** For streaming, traffic is often requested in predictable bursts as the client fills its buffer.
- **Sensitivity to Queuing Delays:** The primary enemy of multimedia quality is not low bandwidth, but **variable queuing delay (jitter)** in the network's routers.
- **Predominantly UDP (for real-time):** Interactive applications prefer UDP to avoid the delays associated with TCP's reliability mechanisms.

Digitizing Audio and Video

To be sent over a network, the analog signals of sound and light must be converted into a digital stream of bits. This involves three key steps:

1. **Sampling:** The continuous analog waveform is sampled at discrete, regular intervals. The **sampling rate** (samples per second, in Hz) determines the fidelity. For example, CD-quality audio is sampled at 44.1 kHz.
2. **Quantization:** The amplitude of each sample is mapped to the nearest value on a discrete numerical scale. The number of bits used for each sample (the **bit depth**) determines the dynamic range. For example, 16-bit audio.
3. **Encoding (Compression):** The resulting stream of numbers is very large. An **encoder (or codec)** is an algorithm used to compress this data, removing spatial and temporal redundancy to drastically reduce the final file size.

Video and Audio Codecs

A **Codec (Coder-Decoder)** is the heart of multimedia networking. It's the algorithm responsible for compression and decompression.

Intra-frame Compression (Spatial): Compresses a single image by removing redundancy within that image. Similar to how a JPEG works. This creates **I-frames (Intra-coded frames)**.

Inter-frame Compression (Temporal): This is where the biggest savings come from. Instead of sending a full new image for every frame, the codec sends only the *differences* between the current frame and the previous one. This creates **P-frames (Predicted frames)** and **B-frames (Bi-directionally predicted frames)**.

Common Codecs:

- **Video:** H.264/AVC (still very common), **H.265/HEVC** (more efficient, used for 4K/UHD), **AV1** (a new, open-source, royalty-free codec gaining popularity).
- **Audio:** MP3, **AAC (Advanced Audio Coding)**, Opus (for real-time communication).

Types of Internet Traffic - Elastic

Traffic on the Internet can be divided into two broad categories: elastic and inelastic.

Elastic Traffic can adjust to changes in delay and throughput across an internet and still meet the needs of its applications.

It is the traditional type of traffic supported on TCP/IP-based internet and is the type of traffic for which internet was designed

Applications that can be classified as elastic are file transfer (File Transfer Protocol [FTP]), e-mail (Simple Mail Transport Protocol [SMTP]), remote login (Telnet, Secure Shell [SSH]), network management (Simple Network Management Protocol [SNMP]), and web access (Hypertext Transfer Protocol [HTTP]).

Types of Internet Traffic - Inelastic

Inelastic traffic does not adapt easily, if at all, to changes in network performance. While elastic applications can tolerate longer delays or temporary reductions in throughput, inelastic applications may become unusable when timing constraints are violated.

Typical examples of inelastic traffic include:

- voice over IP
- video conferencing
- live multimedia streaming
- cloud gaming
- other interactive real-time services

These applications are usually sensitive to one or more of the following QoS dimensions:

- **Throughput:** a minimum transmission rate may be necessary to sustain the media stream
- **Delay (latency):** data must arrive within an acceptable time bound
- **Jitter:** variations in packet arrival time can disrupt smooth playout
- **Packet loss:** some losses may be tolerated, but excessive loss degrades quality significantly

QoS Requirements by Application Class



Voice

One-way latency ≤ 150 ms

One-way peak-to-peak jitter ≤ 30 ms

Per-hop peak-to-peak jitter ≤ 10 ms

Packet loss ≤ 1 percent

Broadcast video

Packet loss ≤ 0.1 percent

Real-time interactive video

One-way latency ≤ 200 ms

One-way peak-to-peak jitter ≤ 50 ms

Per-hop peak-to-peak jitter ≤ 10 ms

Packet loss ≤ 0.1 percent

Multimedia conferencing

One-way latency ≤ 200 ms

Packet loss ≤ 1 percent

Multimedia streaming

One-way latency ≤ 400 ms

Packet loss ≤ 1 percent

Inelastic traffic requirements

The requirements of inelastic traffic are difficult to satisfy in an environment characterized by variable queueing delays, congestion, and packet losses.

This introduces two important needs into the Internet architecture.

First, traffic with stricter requirements may need preferential treatment. Applications such as voice or interactive video cannot always tolerate the same delay and jitter as best-effort traffic. Therefore, the network may need mechanisms to:

- prioritize certain packets
- allocate resources more carefully
- differentiate among traffic classes or flows

Second, these applications may need a way to express their requirements. This can be done:

- explicitly, through reservation or signaling mechanisms
- implicitly, through information carried in the packet headers

At the same time, elastic traffic must still be supported.

Unlike TCP-based applications, many multimedia flows do not naturally reduce their demand during congestion. Therefore, the network must balance multimedia requirements with fairness toward other traffic.

Example: Real-time traffic

Traditional data applications such as file transfer, e-mail, and web access are usually evaluated mainly in terms of throughput and correctness.

If data arrives later, the application may still work acceptably, as long as the information is complete and correct.

Real-time multimedia applications are different because they are also constrained by **time**.

Two common timing models appear in real-time traffic:

- **Constant-rate delivery:**
Data should be delivered at approximately the same rate at which it is generated, so that the receiver can reproduce the stream smoothly.
- **Deadline-based delivery:**
Each unit of data has a useful time window. If a packet arrives after its playback deadline, it may no longer be useful, even if it is delivered correctly.

This is why, in multimedia networking, a packet that arrives too late can be almost as harmful as a packet that is lost.

Example: Real-time traffic

Consider a simple packetized audio application.

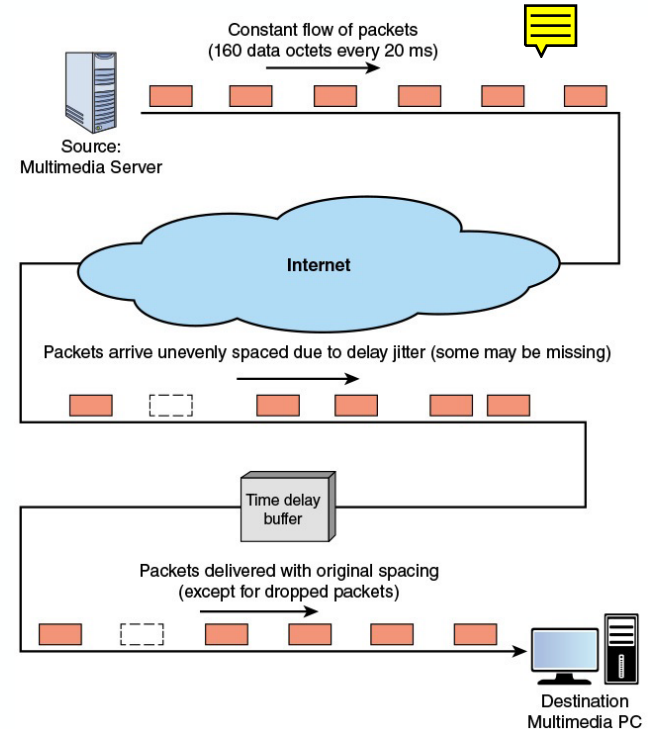
A server generates audio at **64 kbps** and sends it over the network in packets containing **160 octets** of payload, one packet every **20 ms**.

At the sender, packets are generated at regular intervals. However, during transmission over the Internet, the packets may experience different queuing delays. As a result, they do not necessarily arrive at the receiver every 20 ms.

To reconstruct a smooth audio stream, the receiver uses a **playout buffer**:

- arriving packets are temporarily stored
- the receiver delays playback slightly
- packets are then released to the audio application at a constant rate

This buffer absorbs moderate jitter and smooths the stream. However, if a packet arrives too late, the buffer can no longer compensate for it, and the packet must be discarded.



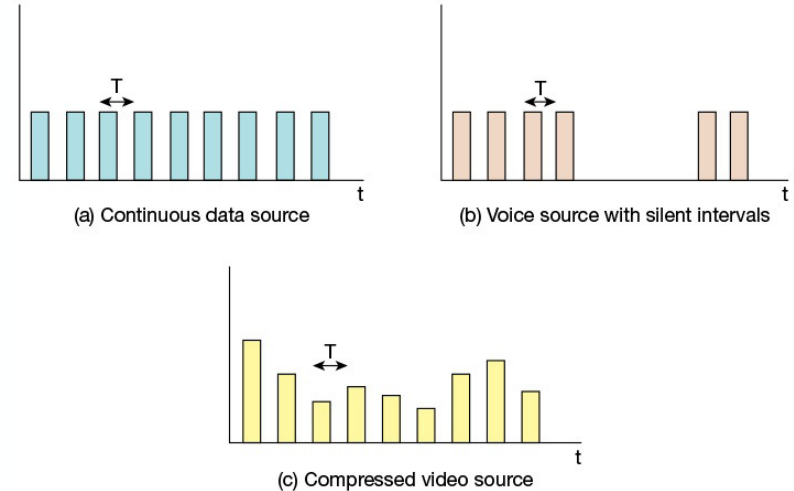
Real-time traffic

Real-time multimedia traffic is not always generated in the same way. Different applications produce packets according to different source models, and this has important implications for buffering, QoS, and transport design.

Three common models are:

- **Continuous data source:** Fixed-size packets are generated at fixed intervals. This is typical of applications that continuously produce media at a stable rate.
- **On/off source:** Periods of packet generation alternate with periods of inactivity. A classic example is conversational voice, where packets are sent during speech activity and silence periods generate little or no traffic.
- **Variable packet size source:** Packets are generated regularly, but their size changes over time. A common example is compressed video, where different frame types produce different amounts of data.

These models matter because the network and receiver must handle not only average traffic, but also burstiness, silence periods, and variable-rate behavior.



Types of Multimedia Services

We can classify multimedia applications into three main categories based on their timing constraints:

1. Streaming Stored Media:

- **Description:** Pre-recorded video and audio (e.g., a movie on Netflix, a video on YouTube).
- **Interaction:** The user can pause, rewind, fast-forward.
- **Key Requirement:** The network must provide enough average throughput to stay ahead of the playback buffer. It is not strictly real-time.

2. Streaming Live Media:

- **Description:** Broadcasting a live event (e.g., a football match, a news broadcast).
- **Interaction:** No user interaction with the timeline.
- **Key Requirement:** Latency from the event to the user display is important (typically a few seconds to a minute).

3. Real-time Interactive Media:

- **Description:** Two-way communication (e.g., VoIP, video conferencing).
- **Interaction:** Conversational and highly interactive.
- **Key Requirement:** **End-to-end latency is the most critical factor.** Jitter must also be very low.

The Rise of Streaming

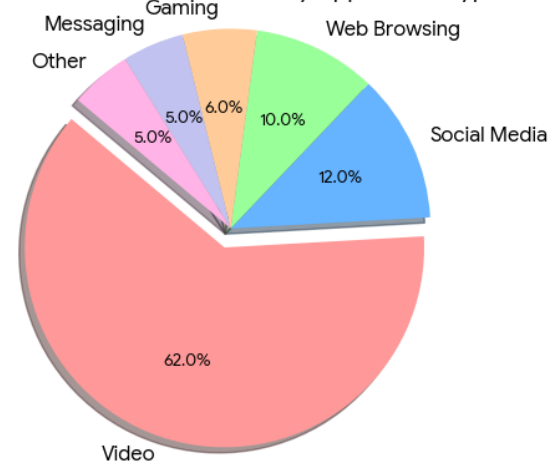
Multimedia streaming is the dominant source of traffic on the internet, and its share continues to grow.

Key Statistics (e.g., from Sandvine Global Internet Phenomena Report):

- Video streaming accounts for over **60%** of all internet downstream traffic volume.
- Major services like Netflix and YouTube are the single largest contributors.
- The growth of 4K content and new immersive formats will continue to drive this trend.

Understanding how to engineer systems that deliver this content efficiently and with high quality is one of the most important topics in modern networking.

Breakdown of Internet Traffic by Application Type



Quality of Service (QoS) and Quality of Experience (QoE)



Network Mechanisms for Quality of Service (QoS)

To support multimedia, network routers can implement QoS mechanisms to treat different types of traffic differently. This is essential for providing a good experience on a congested network.

Key Mechanisms:

1. **Admission Control / Resource Reservation:** In some QoS approaches the network can reserve resources for a specific flow before data transmission begins. If sufficient resources are not available, the flow request may be rejected rather than degrading the service of existing traffic.
2. **Classification and Marking:** Routers first classify incoming packets into different service classes (e.g., "real-time voice," "high-throughput video," "best-effort data"). They then "mark" the packet by setting the **Differentiated Services Code Point (DSCP)** field in the IP header.
3. **Queuing and Scheduling:** Based on the DSCP mark, the router places the packet into one of several queues. A scheduling algorithm then decides which queue to serve next.
 - **Priority Queuing (PQ):** Always serve the highest-priority queue first. Great for voice, but can starve low-priority traffic.
 - **Weighted Fair Queuing (WFQ):** Guarantees each class a certain percentage of the link's bandwidth.



From Best-Effort IP to QoS Architectures

The conventional IP provides the “best-effort” service only, which does not differentiate among different applications.

There have been significant efforts toward data networking with better or even guaranteed QoS, and a representative is the ATM network.

There are two common approaches.

- **IntServ** or *integrated services* is an architecture that *specifies the elements to guarantee QoS in fine-grains for each individual flow*.
- **DiffServ** or *differentiated services* specifies a simple, scalable, *and coarse-grained class-based mechanism* for classifying and managing aggregated network traffic and providing specific QoS to different classes of traffic.



IntServ: Per-Flow QoS with RSVP

Intserv is a QoS model for packet networks that reserves a specified level of service from source to destination.

It was created in the early 1990s by the Integrated Service Group and uses the Resource reSerVation Protocol (RSVP) to reserve necessary resources to support the service requirements requested by the data flow.

There are three Classes of Service (CoS):



- Guaranteed service: strict bounds on the permitted delay and bandwidth.
- Controlled load: same level of service to the application as if the network were completely unloaded. This includes packet loss and delay. Even if the network becomes overloaded, this service class would not be affected.
- Best effort: Traffic is delivered without any guarantee.

IntServ Scalability Burden

The implementation of IntServ has the following requirements:

- Every router must be configured to support RSVP.
- Connection admission control must also be configured throughout the network. This enables the network to reject a connection if sufficient resources are not available.
- Routers must maintain the state and packets per flow of any RSVP connection that passes through it and prioritize traffic accordingly.
- Connection states must be periodically refreshed.
- Packets of a connection flow follow a predetermined path. If a change to a link or router causes rerouting, the flow must re-reserve resources over the new path.

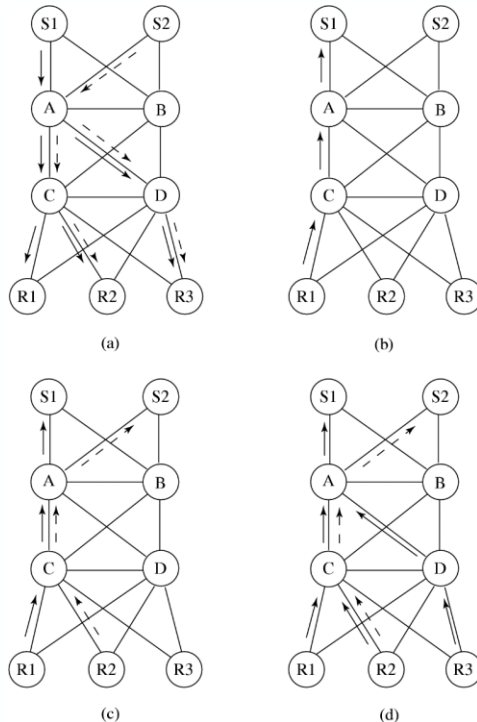
RSVP Essential

In **IntServ**, Flow Specs describe what the resource reservation is for a flow, while the **RSVP** is used as the underlying mechanism to signal it across the network.

RSVP is a setup protocol for Internet resource reservation, which targets a multicast setup for general multimedia applications.

- **RSVP is receiver-initiated** A receiver initiates the reservation request Resv, and the request travels back toward the sender but not necessarily all the way.
- **RSVP creates only soft state** The receiver host must maintain the soft state by periodically sending the same Resv message; otherwise, the state will time out.

A scenario of network resource reservation with RSVP



- S1 and S2 send a *Path* message to the receivers for informing them about the route and the characteristics of the flow without reserving any resource
- R1 sends a **Resv** to S1 for reserving the resources as defined in the *Path* message
- R2 sends a *Resv* to S2 for reserving the resources
 - From A to C two separate channels should be reserved since R1 and R2 request different services
- R2 and R3 send a *Resv* to S1 for reserving the resources
 - The nodes should reconfigure by reserving separate channels or merging equivalent requests

DiffServ: Class-Based QoS in IP Networks

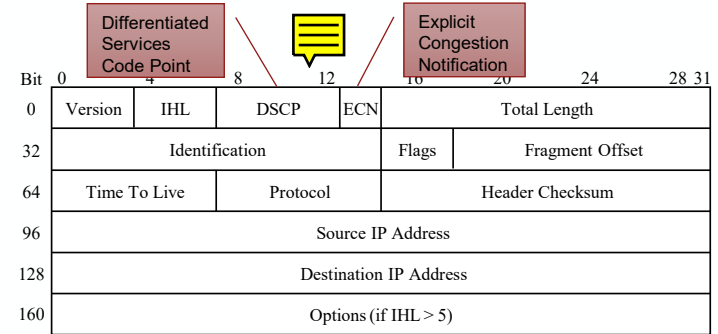
Differentiated Service (DiffServ) operates on a completely opposite approach through the principle of traffic aggregation and classification.

In DiffServ, **network routers implement per-hop behaviors (PHBs), which define the packet-forwarding properties associated with a class of traffic.**

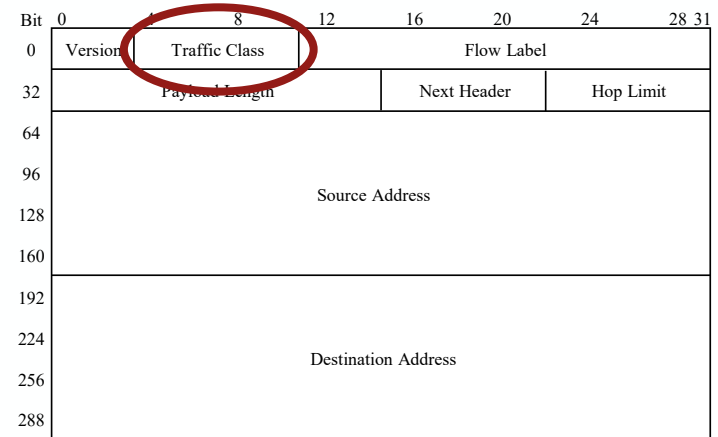
Different PHBs may be defined to five different priorities to different traffic types:

- **Internet traffic** Low priority
- **Multimedia Streaming** Medium Priority
- **Multimedia Conferencing** High Priority

Packet formats of IPv4 and IPv6



(a) IPv4 packet format



(b) IPv6 packet format

ToS/TC marking

The first step for traffic in a DiffServ network is to mark the packets with a value in the DS field

In IPv4, this is the **Type of Service (ToS) field**, and in IPv6, this is the Traffic Class (TC) field.

In any well-designed network, these values are explicitly defined during the design of the QoS implementation.

The traffic is normally marked as it arrives at the first router, the LAN router connected to the source host, or by the host to set during packet creation.

The marking by the router can be made based on a number of different parameters such as source address, destination address, port numbers, or traffic type.

PHB for Multimedia

Different PHBs may be defined to offer different services within a multimedia application data:

- *Uncompressed audio* PCM audio bitstreams can be broken into groups of every n th sample—prioritize and send k of the total of n groups ($k \leq n$) and ask the receiver to interpolate the lost groups if so desired.
- *JPEG image* The different scans in Progressive JPEG and different resolutions of the image in hierarchical JPEG can be given different services.
- *Compressed video* To minimize playback delay and jitter, the best service can be given to the reception of I-frames and the lowest priority to B-frames.

Differentiated Service (Cont'd)

In practice, most networks use the following commonly defined PHB:

- **Default PHB**, which is typically the best-effort service.
- **Expedited Forwarding (EF)**, which is dedicated to low-loss, low-latency traffic.
- **Assured Forwarding (AF)**, which achieves assurance of delivery under prescribed conditions.
- **Class Selector PHBs**, which maintain backward compatibility with non-DiffServ traffic

One implementation may divide network traffic in AF into the following categories and allocate bandwidth accordingly. As an example:

- **Gold**: Traffic in this category is allocated 50% of the available bandwidth.
- **Silver**: Traffic in this category is allocated 30% of the available bandwidth.
- **Bronze**: Traffic in this category is allocated 20% of the available bandwidth.

Example DSCP / PHB Assignments in Practice

PHB	DSCP	DSCP Bin.	Reference	Protocols
EF	EF	101110	RFC 3246	Interactive voice
AF1	AF11	001010	RFC 2597	Bulk transfers, Web, general data
	AF12	001100		
	AF13	001110		
AF2	AF21	010010	RFC 2597	Database, transactions, interactive, preferred data
	AF22	010100		
	AF23	010110		
AF3	AF31	011010	RFC 2597	Locally defined, mission-critical apps
	AF32	011100		
	AF33	011110		
AF4	AF41	100010	RFC 2597	Interactive video and associated voice
	AF42	100100		
	AF43	100110		
IP routing	Class 6	110000	RFC 2474	BGP, OSPF, etc.
Streaming video	Class 4	100000	RFC 2474	Often proprietary
Telephony signaling	Class 3	011000	RFC 2474	SIP, H.323, etc.
Network management	Class 2	010000	RFC 2474	SNMP
Scavenger	Class 1	001000	Internet 2	User-selected service
Other	Default or Class 0	000000	RFC 2474	Unspecified traffic

This table shows example DiffServ mappings used to associate traffic classes with specific DSCP values and Per-Hop Behaviors (PHBs).

The network can assign different forwarding treatments to voice, video, signaling, and best-effort traffic according to their QoS requirements.

You do not need to memorize the code points: the important idea is that packets are grouped into classes, and each class can receive a different treatment in the network.

What Routers Actually Do for QoS

To optimize QoS inside the router while also maximizing throughput on a network interface, routers apply a set of traffic-management functions on packet queues.

Router functions:

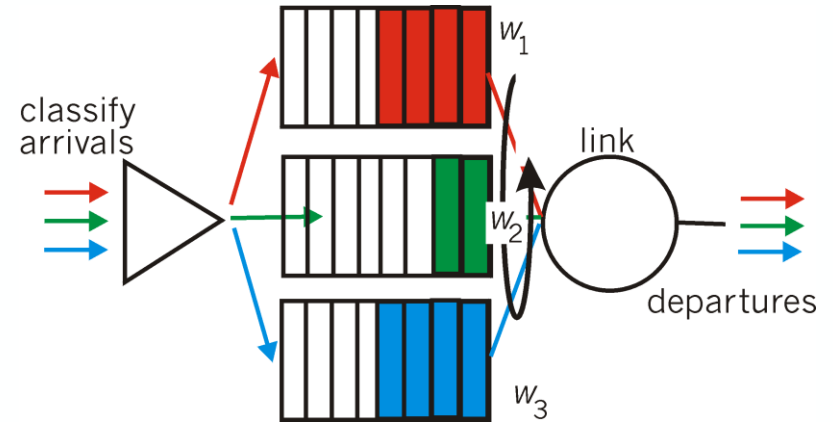
- **Classification and marking:** packets are identified by traffic type and marked so they can receive the appropriate Per-Hop Behavior (PHB).
- **Policing / shaping:** traffic is controlled to enforce the desired rate and service profile; packets may be directed into different queues depending on the QoS policy.
- **Queueing and scheduling:** packets are stored in separate queues and served according to scheduling policies that prioritize or balance traffic classes.
- **Drop policy (e.g., RED):** packets are discarded under congestion conditions; in some cases, packets are dropped before queues are full, as in **Random Early Detection (RED)**.

Weighted Fair Queuing (WFQ) DiffServ

Scheduling mechanism for multiple classes traffic

Each class gets weighted amount of service in each cycle

If w_i is the weight of the i -th class, the related data rate is $R \frac{w_i}{\sum_j w_j}$



DiffServ vs IntServ

Compared with IntServ, DiffServ has coarser control granularity (in aggregated classes, rather than individual flows), and is therefore *simpler* and *scales well*.

However, DiffServ and IntServ are not necessarily exclusive to each other.

In real-world deployment, *IntServ* and *DiffServ* may work together to accomplish the QoS targets with reasonable costs.

A typical scenario is to use IntServ for the edge of the network where individual flows can be managed in a simpler way, while in the core network the DiffServ could be employed by applying the QoS policies to the aggregated flows.

Best-Effort QoS Management

IntServ and DiffServ functions have been implemented in many of today's Internet routers; however, their use in wide area networks remain limited.

- First, the **complexity** of maintaining these services in large-scale dynamic networks can be quite high, particularly for flow-based RSVP;
- Second, the **scale and heterogeneity of Internet terminals and routers make a complete end-to-end QoS guarantee generally difficult**, so for service differentiation.

As such, most of the time, **a networked multimedia application still has to assume that the underlying network is of the best-effort service**, and adaptive transmission and control have to be used through application layer protocols

IntServ, DiffServ, and the Reality of Multimedia Delivery

Approach	Granularity	Guarantee	Mechanism	Complexity	Deployment
Best-effort	All traffic treated equally	None	No Network support	Low	Everywhere
DiffServ	Traffic class	Soft	Packet, marking, scheduling	Medium	Some
IntServ	Per-connection flow	Hard	Packet, marking, call admission	High	Little

In practice, multimedia applications usually assume a largely best-effort Internet and rely on RTP buffering, RTCP feedback, **DASH adaptation**, and CDNs to preserve QoE.

Measuring Success: QoS vs. QoE

How do we measure the quality of a multimedia service? There are two distinct but related perspectives

QoS (Quality of Service):

- **Perspective:** Network-centric.
- **What it measures:** The technical performance of the network itself. These are objective, measurable metrics.
- **Examples:** Bandwidth, packet loss, end-to-end delay (latency), and jitter.

QoE (Quality of Experience):

- **Perspective:** User-centric.
- **What it measures:** The subjective satisfaction and perceived quality of the application from the end-user's point of view.
- **Examples:** Did the video look sharp? Was the call free of annoying echoes or glitches? Did the video buffer?

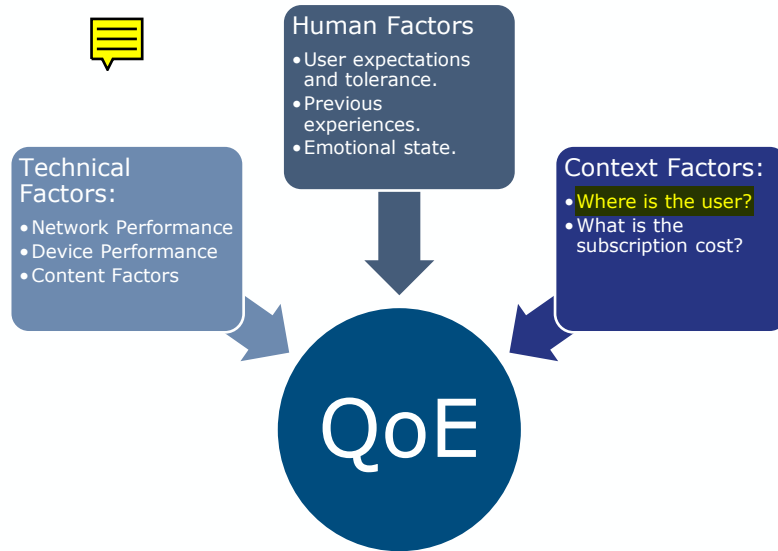
Why QoE is the Ultimate Goal

The Problem with QoS: Perfect network QoS does not always guarantee good user QoE. A user might have a 100 Mbps, zero-packet-loss connection, but if the video server is overloaded and sends a low-quality, pixelated stream, the QoE will be terrible.

The Goal: The ultimate goal of a multimedia service provider is to **maximize user QoE**. They use network QoS as one of the key *tools* to achieve this goal.

QoE drives business success: Users will abandon a service with poor QoE and are willing to pay more for a service with excellent QoE.

What Determines Quality of Experience?



1. Technical Factors:

- **Network Performance:** The underlying QoS (latency, jitter, loss).
- **Device Performance:** The screen size and resolution, CPU power for decoding.
- **Content Factors:** The original encoding quality of the video.

2. Human Factors:

- User expectations and tolerance.
- Previous experiences.
- Emotional state.

3. Context Factors:

- Where is the user? (Watching a movie at home vs. a quick clip on a crowded bus).
- What is the subscription cost? (Users expect higher quality from a paid service).

Quantifying Subjectivity: The Mean Opinion Score (MOS)

Since QoE is subjective, how can we measure it in a standardized way? The most common method is the **Mean Opinion Score (MOS)**.

How it works:

- A group of human subjects are put in a controlled environment.
- They are shown a series of video or audio clips that have been impaired by various network conditions.
- After each clip, they rate its quality on a five-point scale.

The Scale (Absolute Category Rating - ACR):

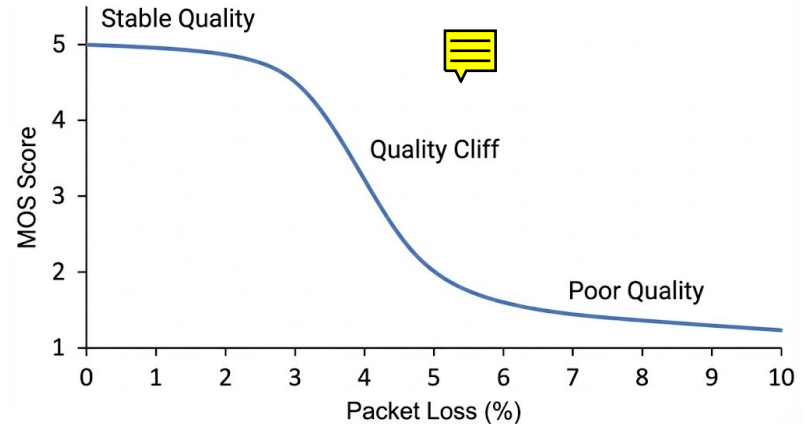
- **5**: Excellent
- **4**: Good
- **3**: Fair
- **2**: Poor
- **1**: Bad

The **MOS** is simply the arithmetic mean of all the scores from all the subjects for a given clip. It provides a single, quantitative number representing the perceived quality.

The Relationship between QoS and QoE

The relationship between a network QoS parameter (like packet loss) and the resulting user QoE (measured in MOS) is typically **non-linear**.

Insight: This shows there are critical thresholds in network performance. The goal of a multimedia system is to always operate in the "flat" top part of the curve and stay away from the cliff.



Predicting QoE: Objective Models (VMAF)

Subjective MOS testing is slow and expensive. **Objective QoE models** are algorithms that try to predict the MOS score by analyzing the video files themselves.

- **PSNR (Peak Signal-to-Noise Ratio)**: A classic, simple metric that measures the mathematical difference between the original and compressed frames. It often correlates poorly with human perception.
- **SSIM (Structural Similarity Index)**: A more advanced model that compares structural information, which is a better predictor of perceived quality.
- **VMAF (Video Multi-method Assessment Fusion)**: Developed by Netflix, this is a state-of-the-art model. It uses machine learning to combine multiple quality metrics to produce a score that is highly correlated with subjective MOS.

Objective Model for Voice: The **ITU-T** E-Model

For voice-over-IP (VoIP), the **ITU-T E-Model (G.107)** is a standard way to predict the voice quality without subjective testing.

It calculates a **Transmission Rating Factor (R-Factor)**, which ranges from 0 to 100. The R-Factor is then mapped to a MOS score.

$$R = R_o - I_s - I_d - I_e + A$$

Where R_o is the base signal-to-noise ratio, I_s represents impairments from the signal itself, I_d represents impairments from **delay and echo**, I_e represents impairments from **packet loss**, and A is an "advantage factor" (e.g., for mobile access).

This model is crucial for network planning and monitoring of VoIP services.

Protocols for Real-Time and Stored Streaming



The Challenge of TCP for Streaming



While TCP provides crucial reliability for data transfer, its mechanisms are often detrimental to time-sensitive multimedia.

1. Head-of-Line (HoL) Blocking:

- If a TCP segment is lost, TCP's in-order delivery guarantee forces the receiver to wait for the lost segment to be retransmitted. All subsequent segments that have already arrived successfully are stuck in the receiver's buffer and cannot be delivered to the application.
- For a video stream, this causes a complete freeze until the retransmission arrives.

2. Congestion Control:

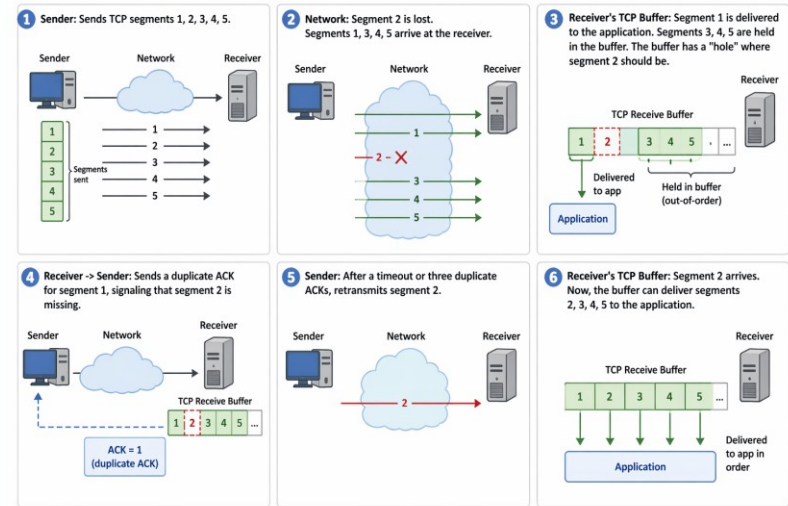
- TCP's congestion control mechanism (slowing down sending rate upon detecting loss) can cause the video bitrate to drop below the playback rate, leading to buffer stalls.

3. Retransmission Delays:

- For a real-time conversation, a retransmitted packet that arrives 200ms late is useless. TCP's insistence on retransmitting this "stale" data wastes bandwidth and adds unnecessary delay.

Visualizing TCP Head-of-Line Blocking

1. **Sender:** Sends TCP segments 1, 2, 3, 4, 5.
2. **Network:** Segment 2 is lost. Segments 1, 3, 4, 5 arrive at the receiver.
3. **Receiver's TCP Buffer:** Segment 1 is delivered to the application. Segments 3, 4, 5 are held in the buffer. The buffer has a "hole" where segment 2 should be.
4. **Receiver -> Sender:** Sends a duplicate ACK for segment 1, signaling that segment 2 is missing.
5. **Sender:** After a timeout or three duplicate ACKs, retransmits segment 2.
6. **Receiver's TCP Buffer:** Segment 2 arrives. Now, the buffer can deliver segments 2, 3, 4, 5 to the application.



The Case for UDP



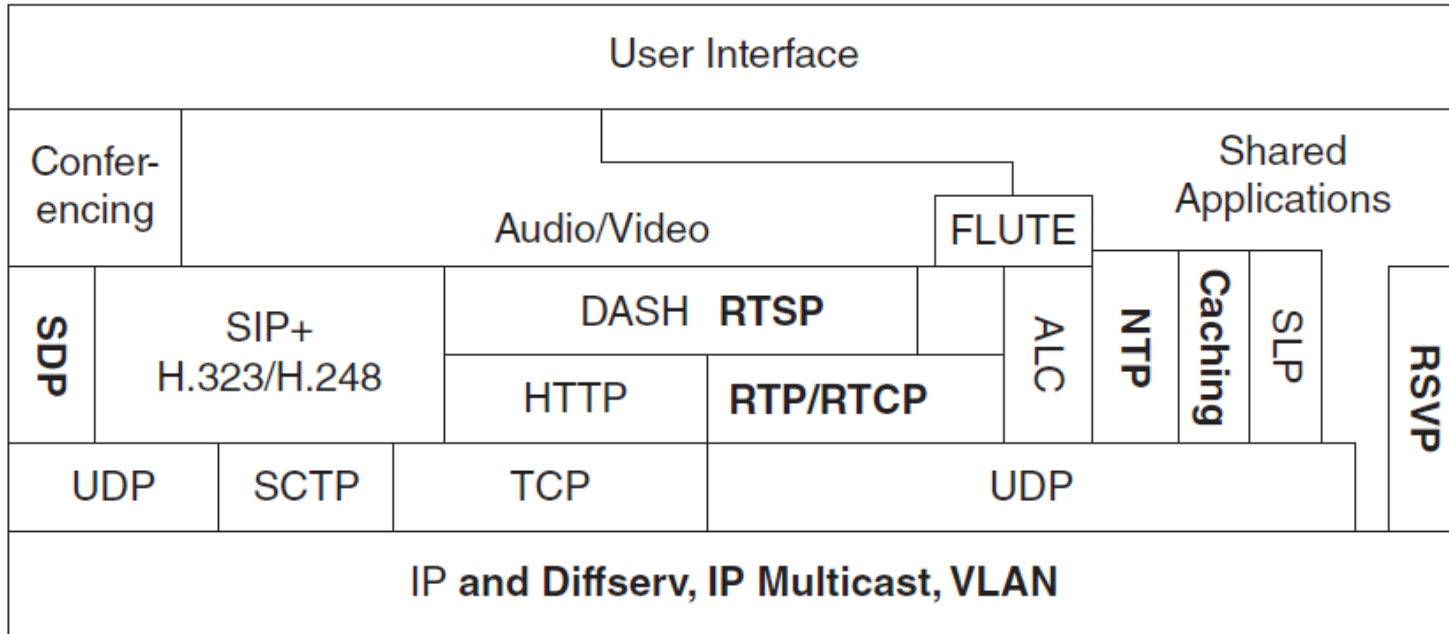
User Datagram Protocol (UDP) provides a minimalist, "fire-and-forget" datagram service.

Why it's a good fit for real-time media:

- **No HoL Blocking:** Each UDP datagram is independent. If datagram 2 is lost, datagrams 3, 4, and 5 are still delivered to the application immediately. The application can choose to play them and conceal the loss of 2.
- **No Delay from Retransmissions:** UDP has no retransmission mechanism. The application gets what arrives, when it arrives.
- **Application-Level Control:** It gives all control over timing, reliability, and ordering to the application layer, which is exactly what a sophisticated multimedia application needs.

The Drawback: UDP provides no help with timing, loss detection, or content identification. That's why we need a protocol like **RTP** to run on top of it.

Multimedia Network Stack



The Protocols of Multimedia

Understanding the challenges of multimedia is the first step. The next is to study the engineering solutions designed to overcome them.

IP networks only provide a "best-effort" service. We need protocols running on top of IP to add the necessary functionality for quality media delivery.

In this block, we will perform a deep dive into the two dominant protocol families:

- **RTP/RTCP**: The workhorse for real-time, conversational media (VoIP, video conferencing).
- **MPEG-DASH**: The state-of-the-art for on-demand and live adaptive streaming (the technology behind Netflix, YouTube, etc.).

Differentiating Protocol Requirements

The protocol design choices are dictated by the application's core requirements.

Real-time Conversational Media (e.g., a video call):

- **Primary Goal:** Minimize **end-to-end latency**. Delay is the enemy of natural conversation.
- **Key Mechanism:** Use **UDP** to avoid retransmission delays. Add timing information to handle jitter at the receiver. The system must be simple and fast.

On-Demand Streaming (e.g., watching a movie):

- **Primary Goal:** Provide a **smooth, continuous, high-quality playback experience** that adapts to changing network conditions.
- **Key Mechanism:** Use **HTTP/TCP** for reliability and firewall traversal. Maintain a **playback buffer** at the client to absorb jitter and network variations. The system must be adaptive and resilient.

Interactive, Live, and Stored Multimedia: A Unified Comparison

Service Type	Main QoS Priority	Typical Stack	Typical Buffering
Interactive media (VoIP, video conferencing)	Very low delay and jitter	UDP + RTP/RTCP + SIP	Very small playout buffer
Live streaming	Low end-to-end delay, continuity	HTTP adaptive streaming + CDN	Small-to-moderate buffer
Stored streaming	Sustained throughput, no stalls	HTTP adaptive streaming + CDN	Larger playback buffer

The more interactive the application, the more the system prioritizes latency over reliability; the less interactive it is, the more buffering and adaptation can be used.

The Real-Time Protocol Stack: RTP over UDP

Interactive applications like VoIP and video conferencing almost always use the **Real-time Transport Protocol (RTP)** running on top of **UDP**.

Why UDP, not TCP?

- **No Head-of-Line Blocking:** In TCP, if one segment is lost, all subsequent segments must wait for it to be retransmitted, even if they have arrived successfully. This would cause a long freeze in a video call. UDP has no such restriction.
- **No Retransmission Delay:** TCP's automatic retransmission mechanism adds unpredictable delay. For a real-time conversation, a packet that arrives 200ms late is useless anyway, so it's better to just skip it than to wait for a retransmission.

The Problem with UDP: UDP itself provides no timing, sequence, or content information. This is where RTP comes in.

RTP (Real-time Transport Protocol) (RFC 3550)



RTP's job is to add the necessary transport-layer functionality for real-time data on top of UDP.

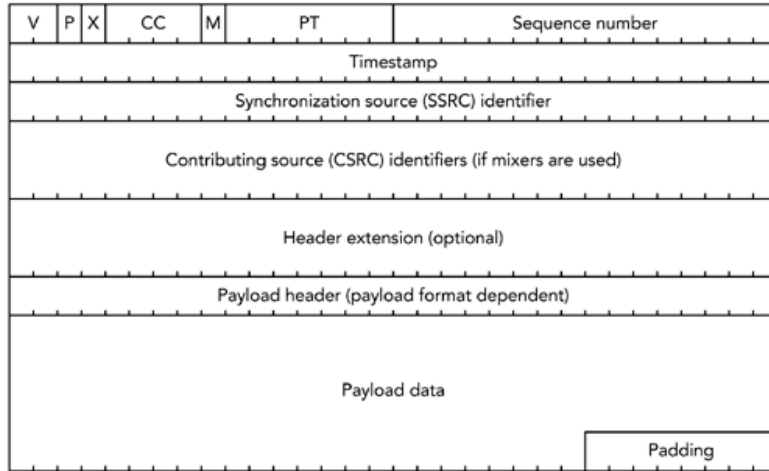
RTP does NOT:

- Guarantee delivery or QoS.
- Have any connection setup or teardown mechanisms.

RTP DOES:

- Provide **sequence numbering** to detect packet loss and reorder packets that arrive out of sequence.
- Provide **timestamps** to allow the receiver to reconstruct the original timing of the media and remove jitter.
- Provide **payload type identification** to specify which codec (e.g., H.264, Opus) is being used.

The RTP Header



V = version number
P = padding
X = extensions
CC = count of contributing sources
M = marker
PT = payload type

The RTP header is added to every single media packet.

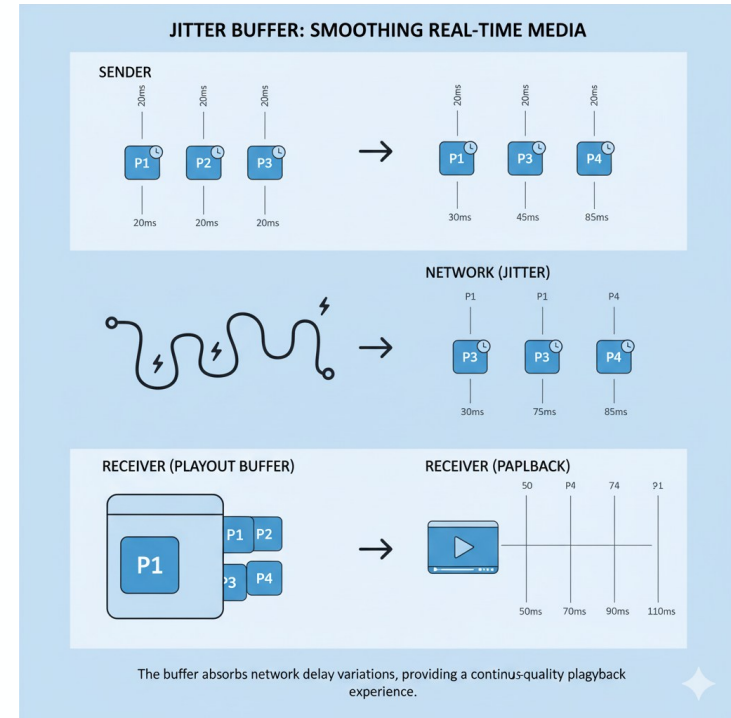
- **Version (V - 2 bits)**: Usually version 2.
- **Padding (P - 1 bit) / Extension (X - 1 bit) / CSRC Count (CC - 4 bits)**.
- **Marker (M - 1 bit)**: Application-specific. For video, it often marks the last packet of a single video frame.
- **Payload Type (PT - 7 bits)**: A number that maps to a specific media codec (e.g., PT 96 might be H.264). This allows for changing codecs mid-stream.
- **Sequence Number (16 bits)**: Increments by one for each RTP packet sent. Used by the receiver to detect loss and for reordering.
- **Timestamp (32 bits)**: Reflects the sampling instant of the first octet in the RTP data packet. All packets from the same video frame share the same timestamp. This is the most important field for jitter calculation.
- **Synchronization Source (SSRC - 32 bits)**: A random ID that uniquely identifies the source of this stream (e.g., your laptop's camera).

How RTP Handles Jitter: The Playout Buffer

Jitter (variable packet arrival time) is the enemy of smooth playback. The receiver uses a **playout buffer (or jitter buffer)** to compensate for it.

1. **Sender:** Sends packets P1, P2, P3, P4 at a constant rate (e.g., every 20ms).
2. **Network:** The network introduces jitter. Packets arrive at the receiver at variable intervals (e.g., P1 at $t=30\text{ms}$, P2 at $t=45\text{ms}$, P3 at $t=70\text{ms}$, P4 at $t=85\text{ms}$).
3. **Receiver (Playout Buffer):** The packets are immediately placed into the buffer upon arrival.
4. **Receiver (Playback):** The receiver's media player pulls packets out of the buffer at a **perfectly constant rate** (every 20ms) and plays them.

Result: The buffer absorbs the network jitter, providing a smooth, continuous stream to the user.



The Playout Buffer Trade-off

The size of the playout buffer is a critical design parameter that involves a direct trade-off.

Large Playout Buffer:

- **Pros:** Can absorb a large amount of jitter, making the playback very resilient to poor network conditions.
- **Cons:** Significantly **increases the end-to-end latency**. The first packet must wait in the buffer before playback can start. This is bad for conversational applications.

Small Playout Buffer:

- **Pros:** Keeps the end-to-end latency very low, which is ideal for conversations.
- **Cons:** If a packet arrives later than the buffer's delay, it is considered lost and must be discarded, resulting in a glitch in the audio/video.

Adaptive Jitter Buffers: Modern systems use adaptive algorithms that dynamically resize the buffer based on real-time measurements of network jitter.

RTCP (RTP Control Protocol)

RTP only handles the one-way transport of media. But how does a sender get feedback on the quality of the stream? How are multiple streams (audio and video) synchronized?

This is the job of the **RTP Control Protocol (RTCP)**.

RTCP is an **out-of-band control protocol** that works alongside RTP. Participants in an RTP session periodically send RTCP packets to everyone else on a separate UDP port.

What is RTCP Used For?

1. **QoS Feedback:** This is the primary function. Receivers send reports back to the sender detailing the quality of the stream they are receiving. This is crucial for rate adaptation.
2. **Session Control:** RTCP packets carry a canonical name (CNAME) that uniquely identifies each participant, even if their SSRC changes (e.g., if their application restarts).
3. **Inter-stream Synchronization:** RTCP provides the timing information needed to synchronize different media streams, for example, to ensure **lip-sync** between an audio and a video stream originating from the same source.

RTCP Packet Type: **Sender Report (SR)**

An SR packet is sent periodically by **active senders** in a session. It provides metadata about the stream it is sending.

Key Fields:

- **SSRC**: The SSRC of the sender.
- **NTP Timestamp**: A 64-bit absolute wall-clock time. This is the crucial field used by receivers to synchronize different streams (e.g., audio and video from the same sender will have SR packets with NTP timestamps that can be correlated).
- **RTP Timestamp**: Corresponds to the same instant as the NTP timestamp.
- **Sender's Packet Count**: Total packets sent.
- **Sender's Byte Count**: Total bytes sent.

RTCP Packet Type: **Receiver Report (RR)**

An RR packet is the essential **feedback mechanism**. It is sent by **receivers** back to the sender and other participants.

Key Fields:

- **SSRC**: The SSRC of the receiver sending the report.
- **Report Block (one per source being listened to)**:
 - **SSRC_n**: The SSRC of the source this report is about.
 - **Fraction Lost (8 bits)**: The percentage of packets lost since the last RR was sent.
 - **Cumulative number of packets lost (24 bits)**.
 - **Extended highest sequence number received**.
 - **Interarrival jitter (32 bits)**: A statistical variance of the packet inter-arrival time, calculated by the receiver. This is a direct measure of network jitter.

How RTCP is Used for Adaptation



The SR and RR packets provide a complete feedback loop.

Workflow:

1. A sender transmits an RTP stream.
2. A receiver receives the stream and calculates the packet loss and jitter.
3. The receiver sends an **RR packet** containing these statistics back to the sender.
4. The sender's application receives the RR. If the reported packet loss or jitter is too high, it can take corrective action, such as:
 - Reducing the video encoding bitrate.
 - Switching to a more error-resilient codec.
 - Increasing the amount of Forward Error Correction (FEC).

Session Control with SIP

RTP and RTCP are excellent for media transport, but they have no concept of "starting a call" or "ending a call." They don't know who to call or how to negotiate which codecs to use.

SIP (Session Initiation Protocol) is the signaling protocol that handles this. It is a text-based protocol similar to HTTP.



SIP's Responsibilities:

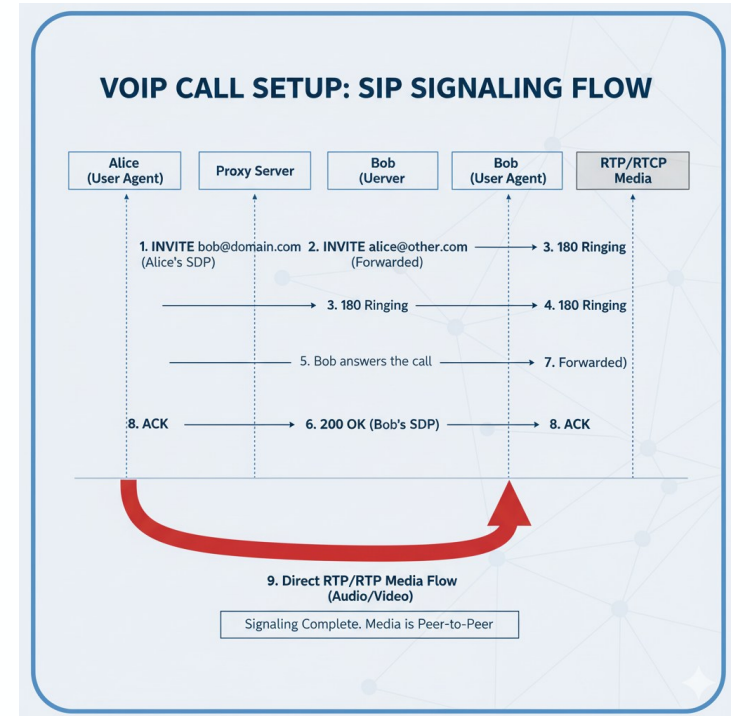
- User Location (finding the IP address of the person you want to call).
- Session Establishment (the "ringing" and "call answered" part).
- Session Negotiation (using the **Session Description Protocol - SDP** to agree on which codecs, IP addresses, and ports to use for the RTP media).
- Session Modification and Termination (placing a call on hold, adding a third person, hanging up).

In practice, SDP carries the media session description, i.e., the codecs, media types, IP addresses, ports, and timing parameters that the two endpoints will use for the RTP streams.

A Simplified SIP Call Setup

A sequence diagram showing two User Agents and a Proxy Server.

1. **Alice -> Proxy:** INVITE bob@domain.com (includes Alice's SDP with her capabilities).
2. **Proxy -> Bob:** INVITE alice@other.com (forwards the request).
3. **Bob -> Proxy:** 180 Ringing (Bob's phone is ringing).
4. **Proxy -> Alice:** 180 Ringing (Alice hears the ringback tone).
5. *(Bob answers the call)*
6. **Bob -> Proxy:** 200 OK (includes Bob's SDP with his chosen codecs).
7. **Proxy -> Alice:** 200 OK (forwards the acceptance).
8. **Alice -> Proxy -> Bob:** ACK (Alice confirms she received the OK).
9. **At this point, the SIP signaling is complete, and the two phones start sending RTP/RTCP media directly to each other using the negotiated parameters.**



The Evolution of On-Demand Streaming

The way we consume stored video over the internet has evolved significantly.

Phase 1: Download and Play: The simplest method. The user downloads the entire video file (e.g., an .mp4) and can only start watching after the download is complete. This is characterized by very long start-up delays.

Phase 2: Progressive Download over HTTP: The video file is placed on a standard web server. The browser starts playing the video as soon as it has downloaded enough data to start.

- **Limitation:** It is not adaptive. If network conditions worsen, the playhead will catch up to the download point, and the video will **stall (buffer)**. There is also no way to seek to a part of the video that hasn't been downloaded yet.

The Breakthrough: HTTP-based Adaptive Streaming (HAS)

Around the late 2000s, a new paradigm emerged that solved the problems of progressive download. This is now the basis for all modern video streaming.

The Three Core Ideas of HAS:

- **Segmentation:** The source video is broken down into many small, independent files or "chunks" (typically 2-10 seconds long) on the server.
- **Multiple Encodings:** Each chunk is encoded at multiple different quality levels (e.g., low-res/low-bitrate up to 4K/high-bitrate).
- **Client-Side Intelligence:** The client (the video player in your browser or app) is responsible for everything. It measures the current network bandwidth and intelligently requests the next chunk at the most appropriate quality level.



Why HTTP Adaptive Streaming Won

HAS succeeded because it brilliantly leverages the existing, massive, and well-understood internet infrastructure.

Firewall & NAT Traversal: It runs over **HTTP (port 80) and HTTPS (port 443)**, which are open on virtually every network in the world. It requires no special protocols or ports, unlike RTP.

Leverages HTTP Caching: It can use the enormous existing ecosystem of HTTP caches, proxies, and **Content Delivery Networks (CDNs)** to bring content closer to users, improving performance.

Adaptive QoE: By dynamically adapting to network conditions, it provides a much smoother user experience with fewer buffering events, maximizing Quality of Experience.

The Dominant Standard: MPEG-DASH

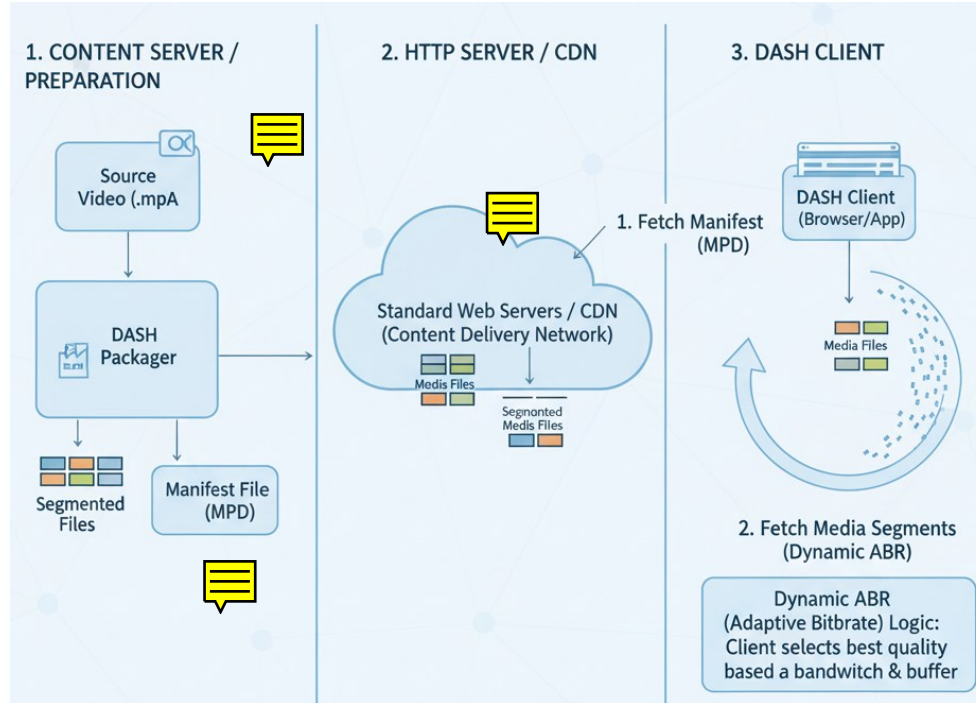
Several companies developed their own proprietary HAS technologies (e.g., Apple HLS, Microsoft Smooth Streaming). To avoid fragmentation, the industry standardized on a single solution.

MPEG-DASH (Dynamic Adaptive Streaming over HTTP) is the international ISO/IEC standard for HAS.

Key Feature: It is **codec-agnostic**. The DASH standard only defines the format of the manifest file and the segment structure. This means it can be used to deliver video compressed with any codec (H.264, H.265, AV1, etc.).

Netflix, YouTube, and most other major streaming services use DASH.

DASH Architecture Overview



Server-Side Content Preparation

Before a video can be streamed, it must be prepared on the server. This is a computationally intensive, offline process.

Step 1: Encoding into Representations:

- The single high-quality source video **is transcoded into multiple versions**, called **Representations**. Each Representation has a different resolution and bitrate.
- **Example:**
 - Rep 1: 1920x1080 @ 8 Mbps
 - Rep 2: 1280x720 @ 4 Mbps
 - Rep 3: 854x480 @ 1.5 Mbps
 - Rep 4: 640x360 @ 0.8 Mbps

Step 2: Segmentation:

- Each Representation is then broken down into small, time-aligned temporal **Segments**. For example, a 10-minute video might be broken into 300 two-second segments. video_1080p_001.m4s, video_1080p_002.m4s, etc.

The Manifest File: Media Presentation Description (MPD)

The **MPD** is the most important part of the DASH standard.

It is an **XML-based manifest file** that describes the entire streaming presentation.

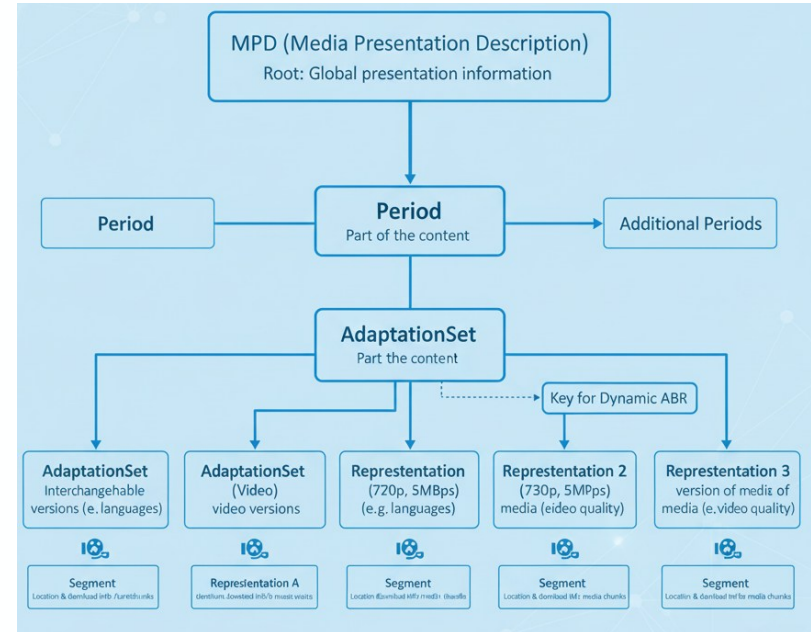
Its purpose: It provides the client with all the information it needs to stream the content, including:

- The different **quality levels** (Representations) that are available.
- The **codecs, resolutions, and bitrates** for each Representation.
- The **URL templates needed to construct the address** for each and every media segment.
- The **duration of the content and the length of the segments.**

The Hierarchical Structure of the MPD

The MPD has a well-defined, hierarchical structure to organize the media content.

- **Visual Suggestion:** A tree diagram showing the hierarchy.
- **MPD (Root):** Contains global information about the entire presentation.
- **Period:** Represents a part of the content with a consistent set of available streams (e.g., the main movie, an advertisement break).
- **AdaptationSet:** A set of interchangeable versions of a single media component (e.g., the video in multiple qualities, or the audio in multiple languages).
- **Representation:** A single, specific encoded version of a media component (e.g., the video at 1080p and 8 Mbps). This is the key element for adaptation.
- **Segment:** Information on how to locate and download the individual media segments for a Representation.



A Simplified MPD Example

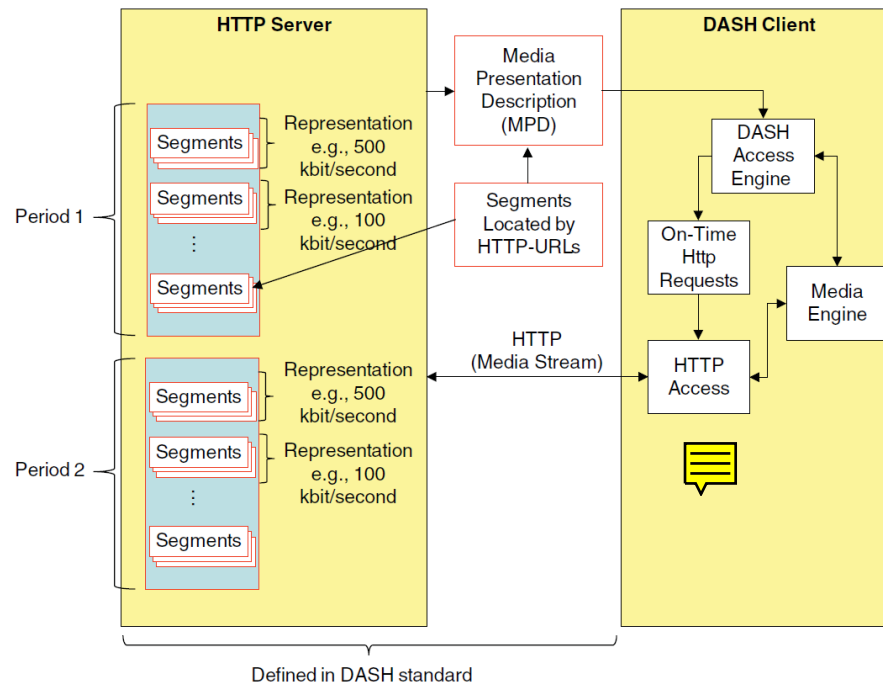
```
<MPD type="static" minBufferTime="PT2S" ...>
  <Period duration="PT600S">
    <AdaptationSet contentType="video/mp4" mimeType="video/mp4" frameRate="25">
      <Representation id="v1" bandwidth="4000000" width="1280" height="720">
        <SegmentTemplate media="video_720p_$Number$.m4s" initialization="video_720p_init.mp4"/>
      </Representation>
      <Representation id="v2" bandwidth="8000000" width="1920" height="1080">
        <SegmentTemplate media="video_1080p_$Number$.m4s" initialization="video_1080p_init.mp4"/>
      </Representation>
    </AdaptationSet>
    <AdaptationSet contentType="audio/mp4" lang="en">
      <Representation id="a1" bandwidth="128000">
        <SegmentTemplate media="audio_en_$Number$.m4s" initialization="audio_en_init.mp4"/>
      </Representation>
    </AdaptationSet>
  </Period>
</MPD>
```

Dissecting the MPD: Key Elements

Let's analyze the key elements from the example.

- **<MPD minBufferTime="PT2S">**: Tells the client it should try to keep at least 2 seconds of media in its buffer.
- **<AdaptationSet contentType="video/mp4">**: This block contains all the available versions of the video stream.
- **<Representation bandwidth="8000000" width="1920" ...>**: Defines a single quality level. The bandwidth attribute is critical; it tells the client that this version requires 8 Mbps.
- **<SegmentTemplate media="video_1080p_\$Number\$.m4s">**: This tells the client how to construct the URL for each segment. The client replaces \$Number\$ with 1, 2, 3, and so on, to fetch the segments in sequence (e.g., video_1080p_1.m4s, video_1080p_2.m4s).

MPEG-DASH Architecture



The DASH Client Logic: The Rate Adaptation Algorithm

The DASH standard defines the MPD and segment formats, but it **intentionally does not standardize the client's rate adaptation algorithm**.

This is the "secret sauce" of a streaming service. It allows for competition and innovation in building the smartest client-side logic.

The Goal of the Algorithm: To select the highest possible video quality that can be sustained without causing the playback buffer to run empty (which would result in a stall).

DASH Client Algorithm: Initialization & Measurement

Step 1: Fetch MPD and Initialization: The streaming session always begins with the client downloading the MPD file. It parses the XML to learn about all the available Representations and how to request their segments. It also fetches the short initialization segment for the Representations.

Step 2: Bandwidth Estimation: The client needs to know the available network throughput. The most common method is to measure the download time of the previous segment.

- $\text{Throughput} = (\text{Size_of_last_segment_in_bits}) / (\text{Download_time_in_seconds})$
- More sophisticated clients will use a moving average over several past segments to get a more stable estimate.

DASH Client Algorithm: Representation Selection

This is the core decision-making step.

Buffer-Based Adaptation: The client's decision is often based on both the estimated bandwidth and the current playback buffer level.

Simple Bandwidth-Based Rule:

- if ($\text{estimated_throughput} > \text{representation_bitrate} * \text{safety_margin}$): Select that representation.
- The client iterates from the highest quality down and picks the first one that satisfies this rule. The safety_margin (e.g., 1.2) is used to be conservative and avoid over-estimating the network capacity.

Buffer-Aware Logic:

- **If the buffer is very full:** The client can be **aggressive** and switch to a higher quality level, as it has a safety cushion if the download is slow.
- **If the buffer is nearly empty:** The client must be **conservative** and may even switch to a lower quality level, even if the bandwidth seems sufficient. The top priority is to **avoid a re-buffering event**.

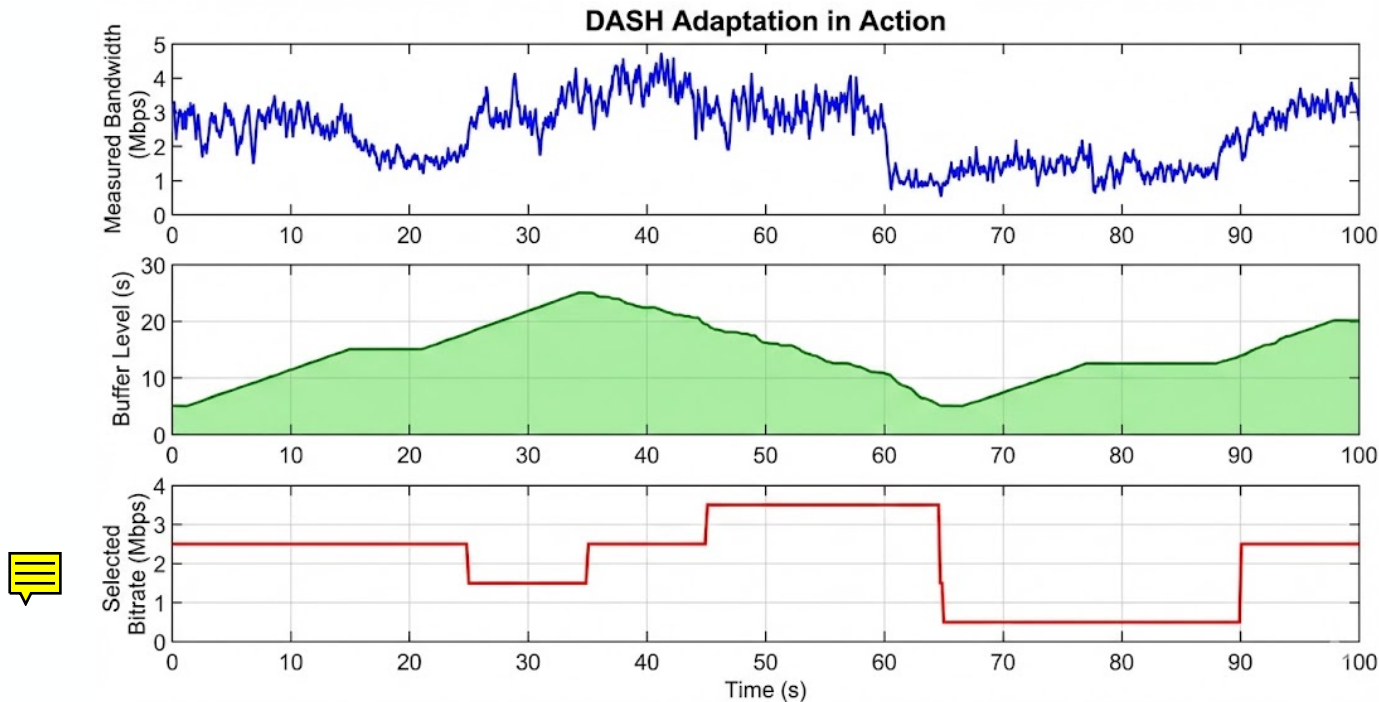
DASH Client Algorithm: Buffer Management & Iteration

The client's primary goal is to keep its playback buffer filled with a few seconds of future video content. The minBufferTime attribute from the MPD provides a hint for the minimum desired buffer length.

The Loop: The client operates in a continuous loop:

- Check the current buffer level.
- If the buffer is below a target level, estimate the bandwidth.
- Select the appropriate Representation for the next segment based on the bandwidth and buffer level.
- Request the next segment from the CDN.
- Once downloaded, append it to the buffer.
- Wait until it's time to download the next segment, and repeat.

DASH Adaptation in Action



MPEG-DASH live streams

There are special considerations when MPEG-DASH is used **for live streams**.

From an end-user viewpoint, the most important consideration is the end-to-end delay.

For example, in the broadcast of a sports event it is annoying to see a certain event (e.g., a goal) with some delays

Another consideration is **the generation and refresh of the MPD**.

- Since the segments are generated in real time, the MPD defines the access URLs either at the end of the broadcast or until the next MPD refresh.
- The client is expected to retrieve the MPD as recommended in the MPD.
- Since the MPD defines the broadcast start time in terms of the wall-clock time, the client's clock **must be synchronized** to properly calculate the latest media segment to request

Content Delivery Networks (CDNs)



Content Delivery Networks (CDNs)

We have studied the protocols for streaming (RTP, DASH), but protocols alone are not enough to deliver a high-quality experience to a global audience.

The physical distance between a user and a server introduces unavoidable latency due to the speed of light. A single server also creates a performance bottleneck.

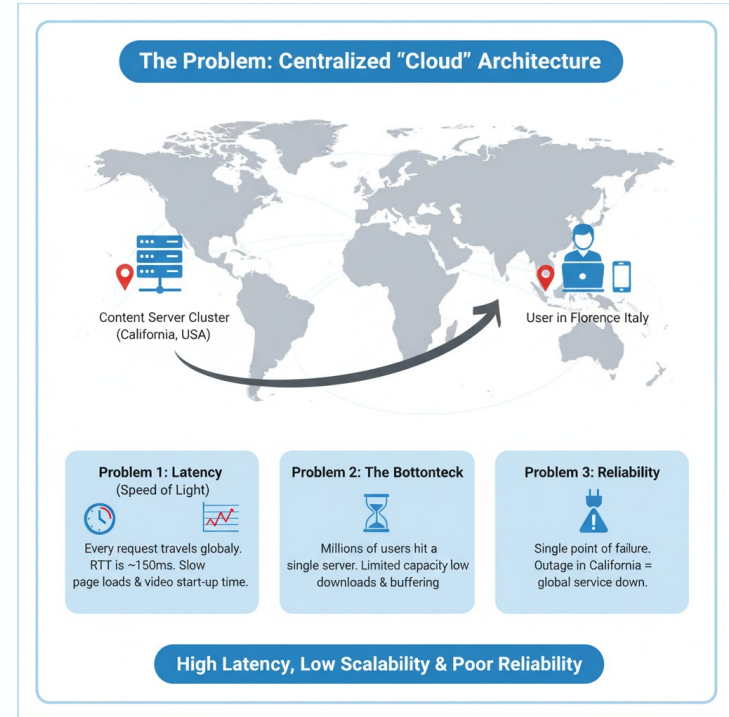
This block provides a deep dive into **Content Delivery Networks (CDNs)**, the massive, geographically distributed systems that solve the problems of latency and scale by moving content closer to the end-user.

The Problem: A Single Origin Server

A single origin server is not sufficient for large-scale multimedia delivery because it creates three major problems:

- **Latency:** users far from the server experience high round-trip times, increasing page-load delay and video startup time
- **Scalability:** a single server and its access links become bottlenecks when many users request the same content
- **Reliability:** failures at the origin can disrupt service for all users

For multimedia services, these problems are particularly severe because video traffic is both bandwidth-intensive and highly sensitive to delay and variability.



The CDN Solution: Move Content to the Edge

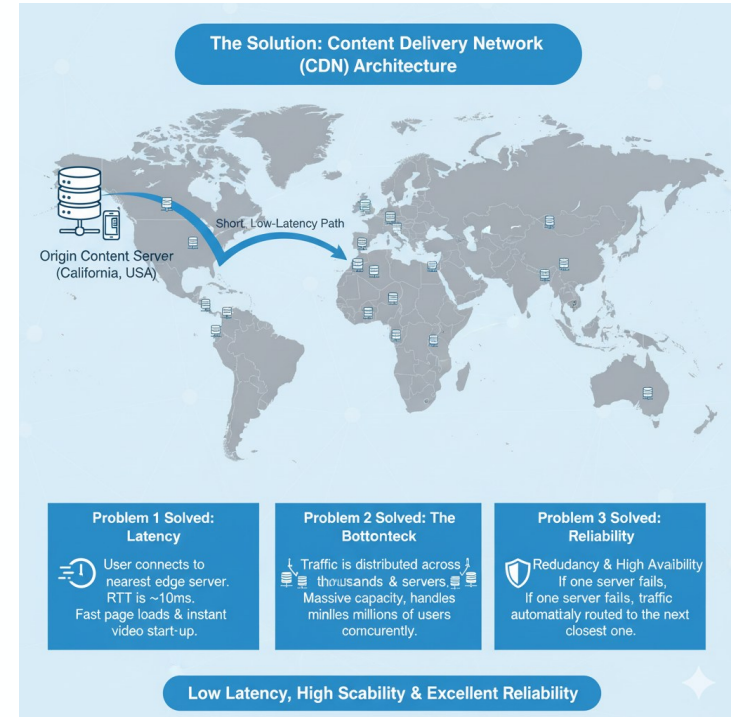
A Content Delivery Network (CDN) solves the origin-server problem by replicating content across many geographically distributed servers.

Instead of always serving content from the origin:

- copies of popular objects are stored at edge servers
- users are directed to a nearby or otherwise suitable replica
- latency is reduced and load is distributed

This approach improves:

- startup delay
- scalability
- resilience
- user QoE



CDN Architecture Overview

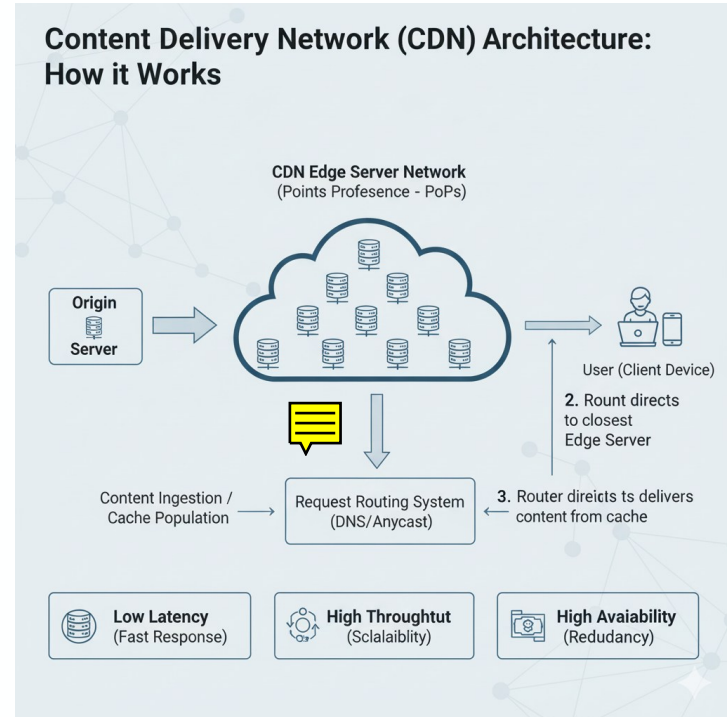
A CDN typically includes three main elements:

- **Origin server:** the authoritative source of the content
- **Edge or surrogate servers:** geographically distributed cache/delivery nodes close to users
- **Request-routing system:** the mechanism that maps each user request to an appropriate edge server

The request-routing system may base its decision on:

- network proximity
- geography
- server load
- content availability
- policy or business constraints

Content Delivery Network (CDN) Architecture: How it Works



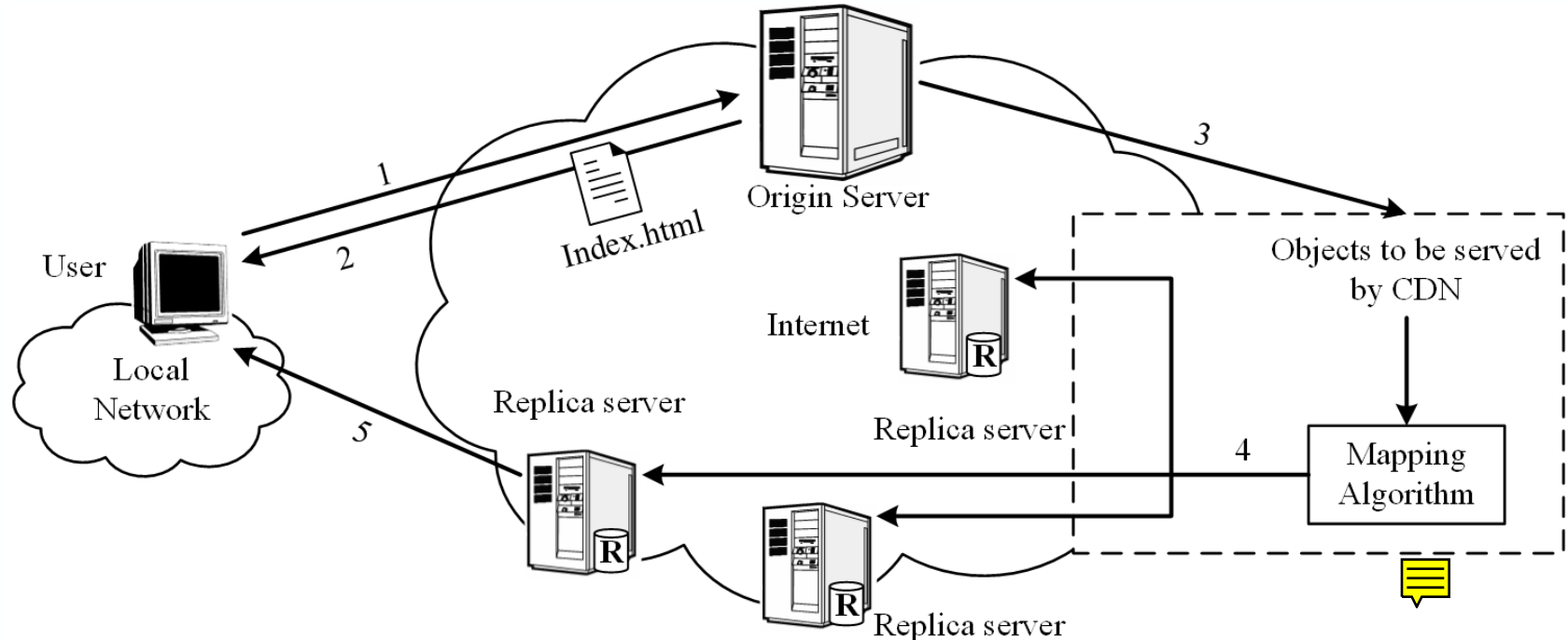
How CDNs Improve Performance & Reliability

CDNs provide several concrete benefits for multimedia delivery:

- **Reduced latency:** shorter paths to users improve response time and startup delay
- **Higher scalability:** traffic is spread across many servers instead of concentrated at the origin
- **Improved availability:** traffic can be redirected if one replica fails
- **Origin offload:** edge caches serve many requests without repeatedly contacting the origin

For streaming, these benefits directly improve QoE by reducing buffering and improving continuity.

High-level request-routing in a CDN



A high-level view of request-routing in a CDN

Overview of Content Distribution Networks (CDNs)

A simplified request-routing process in a CDN is:

1. The user requests content using the provider's URL.
2. The request first reaches the provider's origin infrastructure.
3. The origin or its control logic decides that the content should be served by the CDN.
4. The CDN mapping system selects an appropriate replica server.
5. The selected edge server serves the content to the user.

This process allows the provider to separate:

- service logic at the origin
- content delivery at the edge

The Request Routing Challenge

This is the most complex and interesting part of a CDN.

When a user in their browser requests the resource `http://video.florenceprime.com/movie/segment123.m4s`, how does **the system transparently map** the hostname `video.florenceprime.com` to the IP address of the optimal edge server?

"Optimal" can mean many things:

- The server with the lowest network latency to the user.
- The server that is geographically closest.
- The server that is currently least loaded.
- The server that has the requested content in its cache.

We will explore the two main technical approaches to solve this.

Method 1: DNS-Based Request Routing



The most common method for request routing is to use the **Domain Name System (DNS)**.

The Trick: The content owner (Florence Prime Video) delegates the authority for its video hostname (video.florenceprime.com) to the **CDN's specialized, geographically-aware authoritative DNS server**.

This DNS server is not a standard one; it is a sophisticated piece of software designed to give a different answer depending on *who* is asking.

The DNS-Based Routing Sequence (1/2)

A detailed sequence diagram is essential here.

1. **User Client -> Local DNS Resolver:** The user's device asks its configured local DNS resolver (e.g., their ISP's resolver at IP 8.8.8.8) "What is the IP address for video.florenceprime.com?"
2. **Local DNS Resolver -> Root/TLD Servers:** The local resolver follows the standard DNS hierarchy to find the CDN's authoritative server.
3. **Local DNS Resolver -> CDN Authoritative DNS:** The crucial step. The local resolver sends the DNS query to the CDN's special DNS server.
4. **CDN Authoritative DNS:** This server **sees the IP address** of the machine that sent the query (the user's local DNS resolver).

The DNS-Based Routing Sequence (2/2)

5. **Geolocation:** The CDN DNS server uses a geo-IP database to map the resolver's IP address to a physical location (e.g., "IP 8.8.8.8 is in Florence, Italy").
6. **Server Selection:** Based on this location and its own real-time knowledge of its server loads, the DNS server selects the best edge server (e.g., the server in Milan with IP 1.2.3.4).
7. **CDN Authoritative DNS -> Local DNS Resolver:** The server returns the chosen IP address: "video.florenceprime.com is at 1.2.3.4".
8. **Local DNS Resolver -> User Client:** The client receives the IP address.
9. **User Client -> CDN Edge Server:** The client now makes a direct HTTP connection to the Milan edge server at 1.2.3.4 to download the video segment.

DNS-Based Routing: Pros and Cons

Pros:

- Leverages the existing, highly scalable DNS infrastructure. No special client software is needed.

Cons:

- **Coarse-grained Geolocation:** The CDN only sees the IP of the user's DNS resolver, not the user's actual IP. If a user in Florence is using a DNS server in Rome, they might be directed to a server in Rome instead of a closer one in Milan.
- **DNS Caching:** Intermediate DNS resolvers can cache the response. If the optimal server for a region changes (e.g., due to a failure), it can take time for the old DNS entry to expire from caches, leading to sub-optimal routing.

Improving DNS: EDNS Client Subnet (ECS)

To solve the coarse-grained geolocation problem, an extension to DNS called **EDNS Client Subnet (ECS)** was created (RFC 7871).

How it works: A supporting local DNS resolver, when it queries the CDN's authoritative server, will include the **subnet of the original user's IP address** (e.g., the first 24 bits of the user's IP).

This gives the CDN's DNS server a much **more accurate location** for the end-user, allowing it to make a much better server selection decision.

Major public DNS resolvers like Google DNS and OpenDNS support ECS.

Method 2: Anycast-Based Request Routing

A more advanced and powerful request routing method is to use **IP Anycast**.

Core Concept: Multiple servers in different geographical locations are configured with and announce the **exact same IP address**.

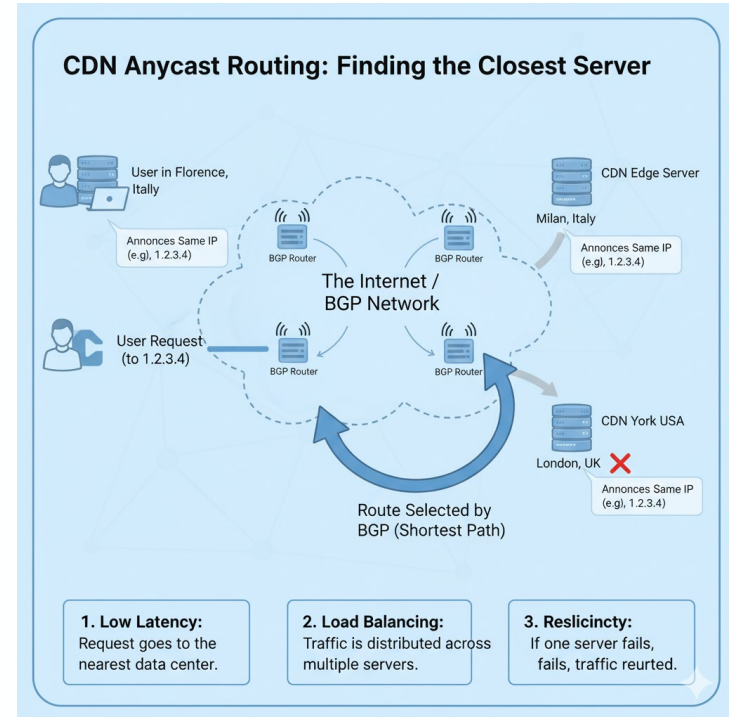
This is not a DNS trick; it's a network routing trick.

How Anycast Works

Each of the CDN's Anycast edge servers uses the **Border Gateway Protocol (BGP)**—the routing protocol of the internet—to announce to its local ISP that it can reach the shared IP address (e.g., 1.2.3.4).

The internet's distributed BGP routing infrastructure will automatically propagate these announcements.

When a user sends a packet to the Anycast IP 1.2.3.4, the BGP routing system will naturally forward the packet along the path with the **lowest network cost (typically the fewest AS-path hops)** to the "closest" server that announced that IP.



Anycast-Based Routing: Pros and Cons

Pros:

- **Extremely Fast and Efficient:** Routing happens at the network layer, which is very fast.
- **Automatic Failover:** If the Milan server fails, it stops announcing the route. BGP automatically reconverges, and traffic is seamlessly redirected to the next-closest server (e.g., London) with no DNS changes needed. Excellent for DDoS mitigation.

Cons:

- **More Complex to Manage:** Requires sophisticated BGP configuration and monitoring.
- **Less Granular Control:** Routing is based on network topology, not server load or application-level metrics.
- **Can be Unstable:** If BGP routes are "flapping" (changing rapidly), users could be bounced between different servers, which can break TCP connections.

The Caching Problem

After choosing the right edge server, the next question is:

How does the content get there?

- A CDN must decide:
- what content to store at the edge
- when to replicate it
- when to fetch it on demand
- how long to keep it cached

This determines both user-perceived performance and origin load.

There are two primary strategies for populating the caches: **Pull and Push.**

Caching Strategy 1: Pull-Based Caching

This is the most common, on-demand, and reactive approach.

The Workflow:

1. A user in Florence requests a video segment (segment123.m4s) for the first time in that region.
2. They are routed to the Milan edge server.
3. The Milan server checks its local cache: it does not have this segment (a **cache miss**).
4. The Milan server forwards the request to the origin server in California to "**pull**" the content.
5. The origin server sends segment123.m4s to the Milan edge server.
6. The Milan server stores a copy of the segment in its local cache and simultaneously forwards it to the user.
7. When the next user in Florence requests the same segment, the Milan server finds it in its cache (a **cache hit**) and serves it directly and instantly.

Time-To-Live (TTL): The content is stored in the cache for a configured amount of time (the TTL), after which it is considered stale and must be re-validated or re-fetched from the origin.

Caching Strategy 2: Push-Based Caching

This is a less common, proactive approach used for predictable, high-demand content.

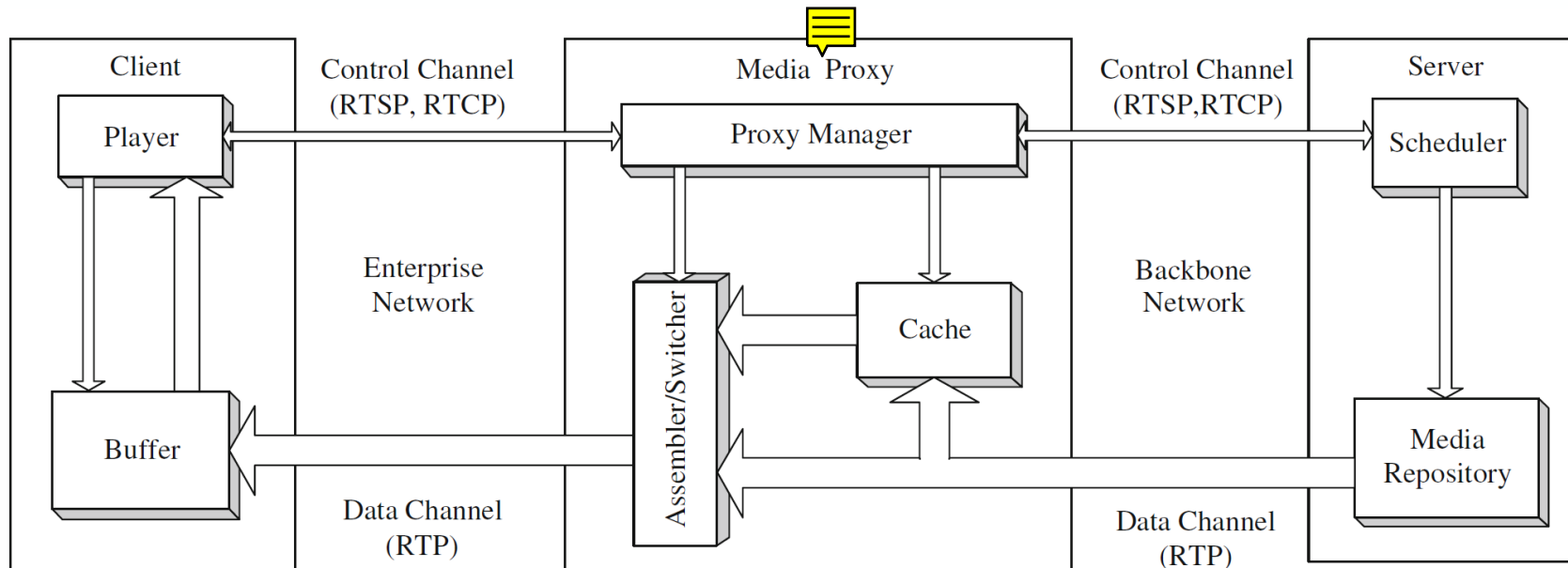
The Workflow:

- The content owner **proactively "pushes"** content to the edge servers *before* it is ever requested by users.
- The origin server sends the new content files to the CDN's storage, and the CDN's management system distributes them to all (or a targeted subset of) the edge servers.

When is this used?

- **Blockbuster Movie Premiere:** Netflix knows that millions of people will try to watch a new episode of a hit show at 9 PM. They will push all the segments to the edge servers hours in advance to handle the massive initial demand.
- **Major Software Update:** Apple or Microsoft will push a new OS update to the CDN edge servers before the release time.
- **Live Event Preparations:** The static assets (logos, player profiles) for a live sports stream can be pushed beforehand.

A generic architecture of a streaming media cache



The CDN Hierarchy

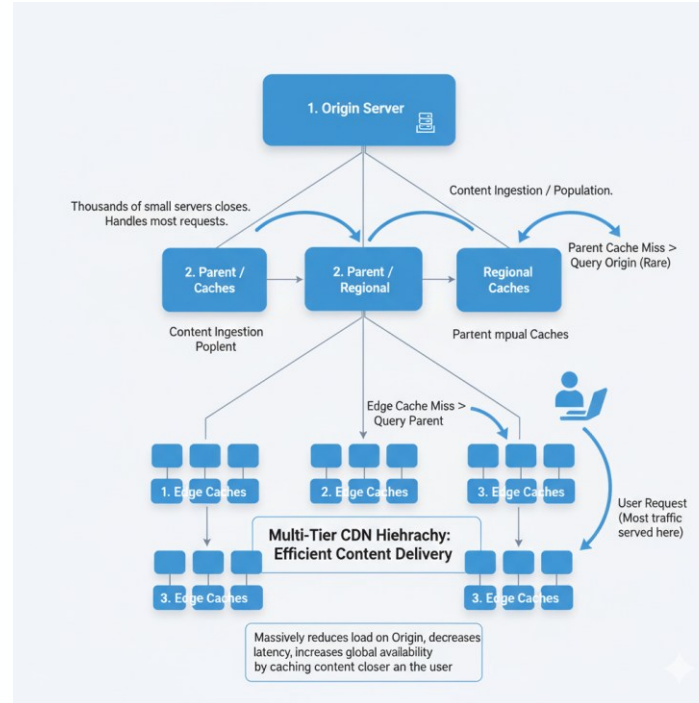
If every single edge server in the world went back to the origin for every cache miss, the origin would still be overloaded. This is known as the "thundering herd" problem.

To solve this, large CDNs are often built as a **multi-tier hierarchy**.

- **Bottom Tier (Edge Caches):** Thousands of small servers in ISP networks, very close to the user. These handle most of the requests.
- **Middle Tier (Parent / Regional Caches):** A smaller number of larger, more powerful cache servers in major internet exchange points.
- **Top Tier (Origin Server):** The single source of truth.

The Flow: If an edge cache in Milan misses, it doesn't go to California. It first asks its parent cache in Frankfurt. The Frankfurt cache likely already has the content because someone in Germany requested it. Only if the parent cache also misses does it go back to the origin. This massively reduces the load on the origin server.

The CDN Hierarchy



Caching Dynamic Content & API Acceleration

While CDNs are famous for caching static files (videos, images), they are also used to accelerate **dynamic content** (content that is personalized for each user, like an API response or an HTML page with your login name).

How can you cache dynamic content? You can't, directly. But the CDN can still help significantly.

Mechanisms:

- **Route Optimization:** The CDN uses its sophisticated network monitoring to find the most reliable and least congested internet path from the edge server back to the origin server, avoiding internet traffic jams.
- **TCP Connection Optimization:** The user makes a short, potentially unreliable connection to the nearby edge server. The edge server maintains a long-lived, highly optimized, persistent TCP connection back to the origin. The edge server acts as a stable proxy.

This is often called **API Acceleration**.

The Perfect Marriage: DASH + CDN

HTTP Adaptive Streaming (like DASH) and CDNs are a perfect technological match. They were practically made for each other.

Why they work so well together:

- **Cache-Friendly:** DASH breaks content into small, independent, static files (the segments). These are ideal for caching. The same segment (video_1080p_123.m4s) can be served to thousands of different users.
- **Leverages Request Routing:** The DASH client makes a separate HTTP request for every single segment. The CDN's request routing system ensures that each of these independent requests is sent to the optimal, low-latency, high-bandwidth edge server.

It is this combination of client-side intelligence (DASH) and network-side intelligence (CDN) that powers the modern streaming internet.

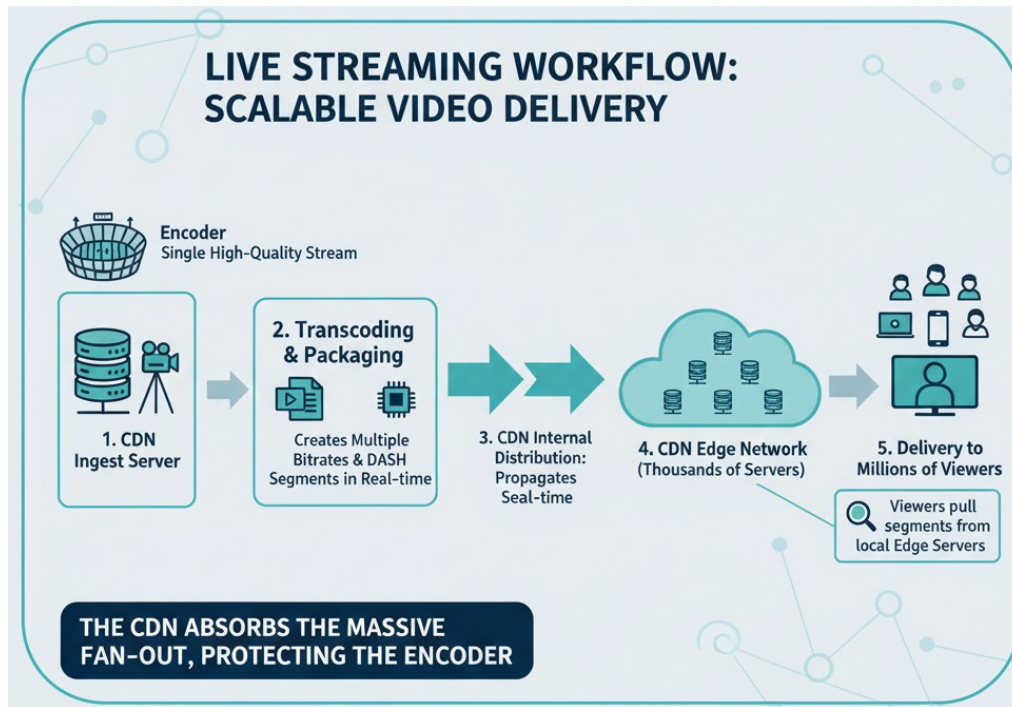
The Role of the CDN in Live Streaming

CDNs are even more critical for live streaming, where a single source must be delivered to millions of concurrent viewers.

1. **Encoder (at the event):** A single encoder at the football stadium sends one high-quality stream to the CDN.
2. **CDN Ingest:** The stream is received by a single **ingest server** in the CDN's network.
3. **Transcoding & Packaging:** The ingest server (or a connected media server) transcodes the stream into multiple bitrates and packages it into DASH segments in real-time.
4. **Distribution:** The CDN's internal distribution network propagates these newly created segments instantly across its thousands of edge servers.
5. **Delivery:** Millions of viewers are now all pulling the segments from their respective local edge servers.

The Key: The CDN absorbs the massive fan-out, protecting the single encoder at the event from being overwhelmed.

The Role of the CDN in Live Streaming



Case Study: NetFlix



Case Study: Netflix

Netflix is a particularly important case study in multimedia networking because it combines:

- extremely high traffic volumes
- large and highly variable user demand
- global content delivery
- the need for continuous, high-QoE streaming

Historically, Netflix relied heavily on cloud infrastructure and **third-party CDN providers** to deliver its service at scale.

In the earlier phase of its architecture, Netflix used Amazon Web Services for:

- **content conversion:** creating multiple encoded versions of each title
- **content storage:** storing the encoded assets
- **content distribution:** transferring content to CDN infrastructures that streamed it to end users

This case study illustrates the evolution from reliance on third-party cloud/CDN infrastructure to Netflix's own optimized delivery platform, Open Connect.

Case study: Netflix

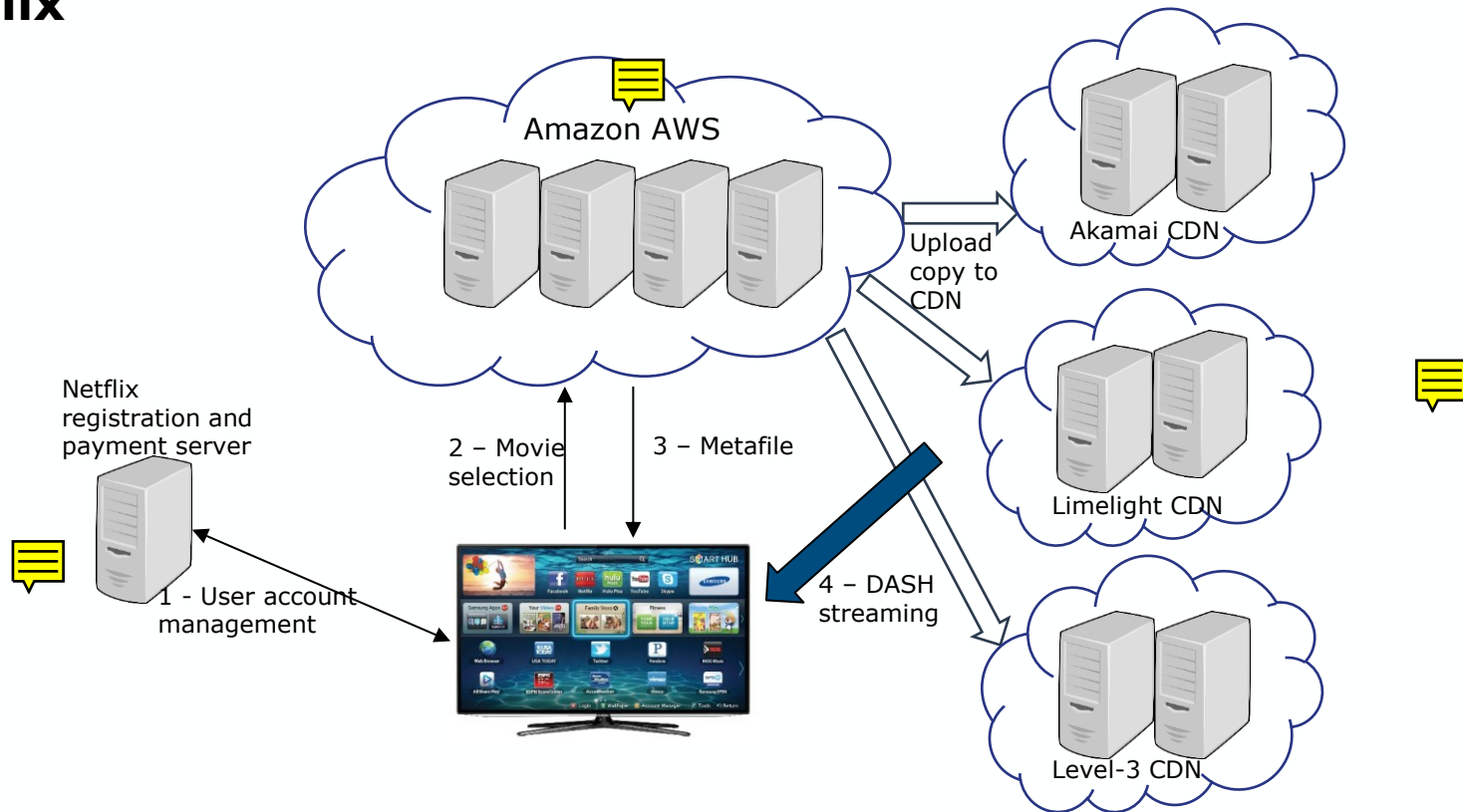
In its earlier architecture, Netflix owned relatively little delivery infrastructure directly and relied on a combination of:

- its own user-facing service components, such as registration and payment servers
- Amazon cloud services for storage and transcoding
- third-party CDNs for large-scale video delivery

A simplified flow was:

1. Netflix uploaded studio masters to the cloud
2. Multiple versions of each movie were created with different encodings and quality levels
3. These versions were distributed to CDN providers
4. The CDNs streamed the selected version to end users
5. The cloud also hosted parts of the Netflix web service used for browsing and selection

Netflix



Netflix OpenConnect

As Netflix traffic kept growing, the company progressively introduced its own content delivery infrastructure, called **Open Connect**.

Open Connect is Netflix's dedicated CDN, designed specifically to bring video content closer to end users through HTTP/HTTPS delivery.

This shift reflects an architectural evolution:

- from stronger dependence on generic cloud + third-party CDNs
- toward a delivery system optimized specifically for Netflix traffic patterns

Open Connect shares the basic goals of other CDNs:

- reduce latency
- reduce upstream bandwidth usage
- improve scalability
- increase delivery efficiency

But it is more tightly integrated with the needs of a single large streaming provider.

Why OpenConnect

Netflix launched the Open Connect initiative in response to the rapidly increasing scale of its streaming traffic.

Two major motivations were:

- **Closer collaboration with ISPs:**

As Netflix became a significant fraction of traffic on many access networks, it became important to work directly with ISPs to improve delivery efficiency and reduce congestion.

- **More efficient content placement:**

A Netflix-specific CDN could move popular content closer to users in a proactive way, reducing the need to repeatedly fetch content across expensive upstream links.

Compared with a purely demand-driven CDN approach, this model allows:

- better traffic engineering
- lower upstream bandwidth demand
- more direct control over placement and delivery

Netflix OpenConnect

The core building blocks of Open Connect are **Open Connect Appliances (OCAs)**.

An OCA is a specialized server appliance that:

- stores encoded video and image assets
- serves them to client devices via HTTP/HTTPS
- is optimized for fast, high-volume video delivery

Client devices may include:

smart TVs

- set-top boxes
- mobile devices
- computers

The main objective of an OCA is simple:

deliver Netflix content to users as quickly and reliably as possible from a location that is as close as feasible to the end user.

Netflix OpenConnect

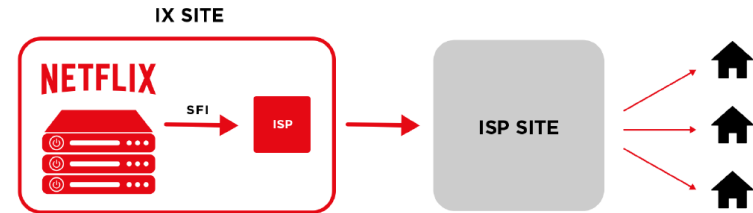
OCAs can be deployed at major **Internet Exchange Points (IXPs)** in important Netflix markets around the world.

In this model:

- OCAs are positioned at highly connected locations in the Internet
- multiple ISPs can reach them efficiently through public or private peering
- traffic can be delivered with fewer costly long-distance traversals

This provides important benefits:

- reduced latency
- reduced transit costs
- improved scalability
- better performance for multiple ISP networks at once



Netflix OpenConnect

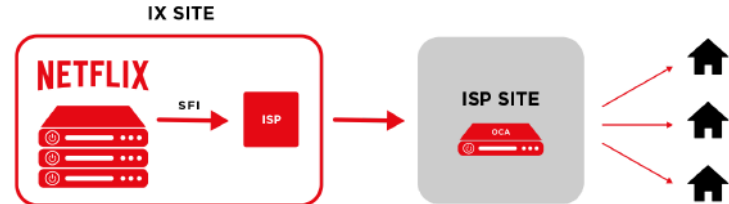
In addition to IXP deployment, Netflix can also provide OCAs directly to qualifying ISPs.

In this model:

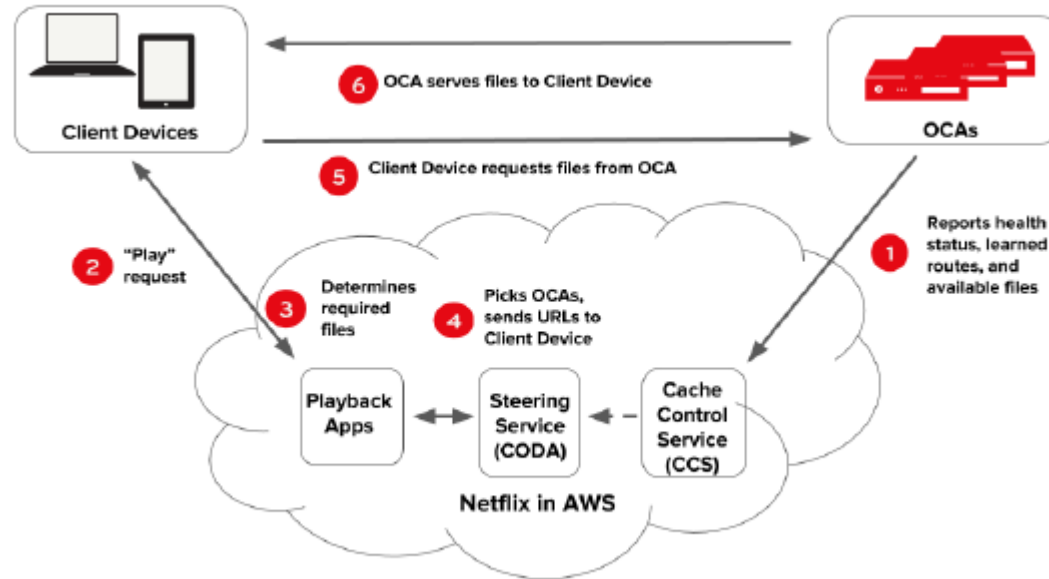
- Netflix provides the server hardware
- the ISP provides power, space, and connectivity
- the appliances are installed inside the ISP network itself

This gives further advantages:

- content is brought even closer to subscribers
- upstream traffic is reduced further
- the ISP can directly control how local users are routed to the embedded OCAs
- peering and nightly updates can still be used for resilience and content refresh



Netflix OpenConnect



<https://openconnect.netflix.com/>

Netflix OpenConnect



Numerical Examples & Future Trends



Multimedia Networking in Practice

We have explored the foundational concepts, protocols, and systems that deliver multimedia.

In this final block, we will make these concepts quantitative:

- **Numerical Example 1:** We will calculate the bitrate of raw and compressed video to understand the scale of the data involved.
- **Numerical Example 2:** We will simulate the decision-making process of a DASH client to see how it adapts to changing network conditions.

We will then look ahead to the future trends and open challenges that will define the next generation of multimedia networking.

Numerical Example 1: Video Bitrate Calculation

The Goal: Understanding Video Data Rates

It is difficult to appreciate the engineering challenges of video streaming without first understanding the sheer volume of data involved in uncompressed video.

This example will demonstrate why video compression (codecs) is not just an optimization, but an absolute necessity for streaming over any practical network.

We will calculate the bitrate for a standard Full HD video stream.

Uncompressed Bitrate Calculation

Let's calculate the raw bitrate for a typical video with the following parameters:

- **Resolution (1080p):** 1920 pixels (width) x 1080 pixels (height) = 2,073,600 pixels per frame.
- **Color Depth:** We use 24 bits per pixel (8 bits for each color channel: Red, Green, and Blue) for true color.
- **Frame Rate:** 30 frames per second (fps). This is standard for most video content.
- **Calculation:**
 - **Bits per frame** = 2,073,600 pixels/frame * 24 bits/pixel = 49,766,400 bits/frame.
 - **Bitrate (bits per second)** = 49,766,400 bits/frame * 30 frames/sec = 1,492,992,000 bps.
- **Result: ~1.49 Gbps.**

Streaming uncompressed Full HD video requires a dedicated 1.5 Gbps connection. This is completely impractical for the vast majority of internet users. For 4K video, the number is over 6 Gbps.

The Solution: Video Compression

As we discussed earlier, video **codecs** (like H.264, H.265) are used to compress this massive data stream.

They do this by removing two types of redundancy:

- **Spatial Redundancy**: Redundancy within a single frame (e.g., a large area of blue sky).
- **Temporal Redundancy**: Redundancy between consecutive frames (e.g., the background is identical between frame N and frame N+1).

We can measure the effectiveness of a codec by its **compression ratio**.

Compression Ratio = (Uncompressed Size) / (Compressed Size)

Typical Codecs and Compression Ratios

The compression ratio is not fixed; it depends on the complexity of the video content. A simple "talking head" video compresses much more than a high-action scene. However, we can use typical, rule-of-thumb values.

H.264 / AVC (Advanced Video Coding):

- Still the most widely supported codec on the web.
- Typical Compression Ratios: **50:1 to 500:1**.

H.265 / HEVC (High Efficiency Video Coding):

- The successor to H.264, used for 4K/UHD and HDR content.
- Provides roughly **40-50% better compression** than H.264 for the same perceived visual quality.
- Typical Compression Ratios: **100:1 to 1000:1**.

AV1:

- A new, open-source, royalty-free codec developed by the Alliance for Open Media (Google, Netflix, Amazon, etc.).
- Aims to provide **~30% better compression** than HEVC. Its adoption is growing rapidly.

Compressed Bitrate Calculation

Let's re-calculate the bitrate for our 1.49 Gbps uncompressed stream using a moderate compression ratio of **300:1**, which is typical for H.264.

Calculation:

- **Compressed Bitrate** = Uncompressed Bitrate / Compression Ratio
- Compressed Bitrate = 1,492,992,000 bps / 300 = 4,976,640 bps.

Result: ~5.0 Mbps.

Through compression, we have reduced the required bandwidth from 1.5 Gbps to a much **more manageable 5 Mbps**, making it streamable over a typical home internet connection. This demonstrates that **the codec is as important as the network itself**.

The Bitrate Ladder in a DASH Manifest

A DASH manifest file (the MPD) contains a list of available Representations, each with a different resolution and target bitrate. This is often called a **bitrate ladder**. The client's job is to intelligently climb up and down this ladder.

Representation ID	Resolution	Codec	Target Bitrate	Required Bandwidth
Rep 1	640 x 360	H.264	800 kbps	~1 Mbps
Rep 2	854 x 480	H.264	1.5 Mbps	~2 Mbps
Rep 3	1280 x 720	H.264	4.0 Mbps	~5 Mbps
Rep 4	1920 x 1080	H.264	8.0 Mbps	~10 Mbps
Rep 5	3840 x 2160	H.265	16.0 Mbps	~20 Mbps

Note: The "Required Bandwidth" is typically higher than the target bitrate to provide a safety margin.

Numerical Example 2: DASH Client Simulation

The Scenario: A DASH Client in a Mobile Environment

The Client: A user is watching a video on their smartphone while on a train.

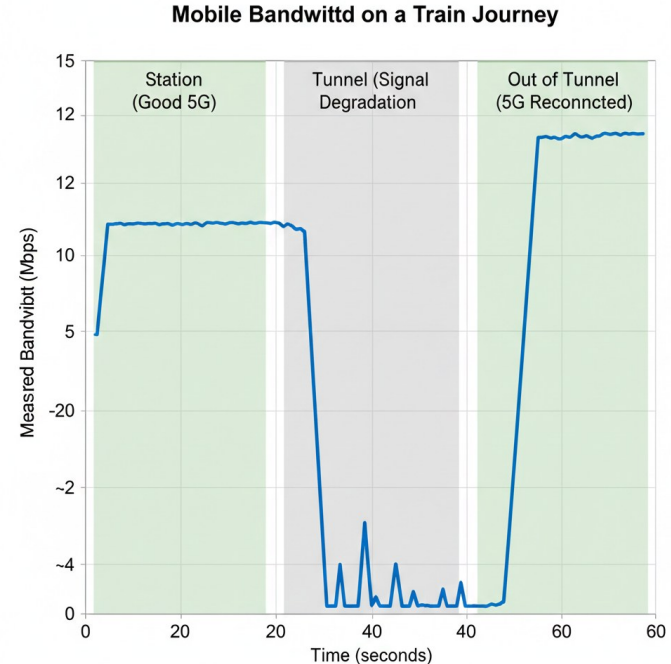
The Content: The video is available using the bitrate ladder from the previous slide. The segment duration is 4 seconds.

The Goal: To simulate the decisions the client's rate adaptation algorithm makes as the network quality fluctuates during the train journey.

The Dynamic Network Conditions

The client continuously measures its available bandwidth by timing segment downloads.

- **0-20s:** The train is in a station with good 5G coverage. The bandwidth is high and stable at **~12 Mbps**.
- **20-40s:** The train enters a tunnel. The signal degrades significantly. The bandwidth drops sharply to **~3 Mbps**.
- **40-60s:** The train exits the tunnel and reconnects to 5G. The bandwidth quickly recovers to **~15 Mbps**.



The Client's Decision Process

Let's trace the client's logic at key moments.

t = 10s (In the station):

- **State:** Measured Bandwidth ≈ 12 Mbps. **Playback buffer is full** (e.g., 30s).
- **Decision:** 12 Mbps is greater than the 10 Mbps required for the 1080p stream. The buffer is healthy.
- **Action:** **Request the 1080p / 8.0 Mbps representation** for the next segment.

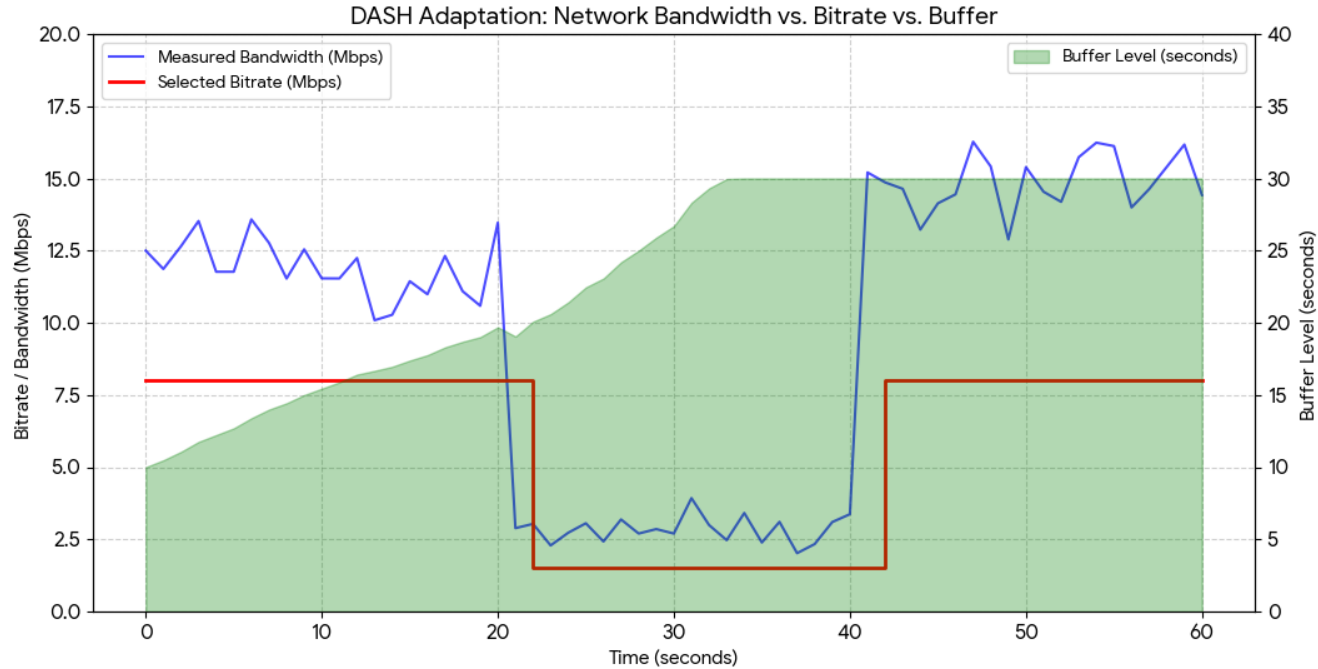
t = 30s (In the tunnel):

- **State:** Measured Bandwidth ≈ 3 Mbps. Buffer is draining quickly because the download speed is slower than the playback speed. Buffer level is now low (e.g., 6s).
- **Decision:** 3 Mbps is not enough for the 720p stream, but it is enough for the 480p stream (which needs ~ 2 Mbps). The low buffer makes avoiding a stall the top priority.
- **Action:** **Request the 480p / 1.5 Mbps representation.** The user sees a temporary drop in quality, but the video keeps playing.

t = 50s (Exiting the tunnel):

- **State:** Measured Bandwidth ≈ 15 Mbps. Buffer has started to refill.
- **Decision:** The bandwidth is now high enough to support the 1080p stream again.
- **Action:** **Switch back up to the 1080p / 8.0 Mbps representation.**

Visualizing the Result



Future Trend: Low Latency DASH (LL-DASH)

A major drawback of traditional DASH for live events is the high latency (typically 30-45 seconds behind the live action). This is unacceptable for sports and interactive events.

Low Latency DASH (LL-DASH) is a new extension to the standard designed to reduce this latency to **~2-5 seconds**.

Key Technical Mechanisms:

- **Chunked Transfer Encoding:** The server uses HTTP chunked transfer to send a segment in smaller "chunks" *as it is being created*, rather than waiting for the full segment to be finished.
- **Shorter Segments:** Using very short segments (e.g., <1 second) to reduce the minimum unit of delay.
- **Manifest Updates:** The manifest is updated frequently to announce the availability of new segments as soon as they are ready.

Future Challenge: VR / 360-degree Video Streaming

Streaming high-quality, interactive 360-degree video for VR is a monumental challenge.

The Bandwidth Problem: A full 360-degree 8K video stream can require over 100 Mbps, which is not feasible for most mobile users.

The Inefficiency: At any given time, the user is only looking at a small portion of the 360-degree sphere (the **viewport**). Sending the entire sphere in high quality is incredibly wasteful.

The Solution: Viewport-Dependent Streaming:

- The client tracks the user's head orientation.
- It requests only the tiles of the video corresponding to the current viewport in high quality.
- It requests the rest of the sphere (the periphery) in a very low base quality layer.

This requires ultra-low latency to react to fast head movements.

The Role of MEC in Interactive Multimedia

Mobile Edge Computing is the perfect platform to solve these emerging challenges.

For LL-DASH: A local MEC server can act as a CDN cache for live stream segments, minimizing the RTT and helping to stabilize latency.

For VR Streaming: The MEC server can host the computationally intensive **viewport processing logic**. Instead of the client requesting individual tiles, it sends its head orientation to the MEC. The MEC then fetches the correct tiles, stitches them together into a final frame, and streams that frame to the user. This simplifies the client and reduces signaling overhead.

For Cloud Gaming: The MEC server can host the entire game engine, providing the sub-20ms latency required for an interactive experience.

The Impact of AI/ML on Multimedia Delivery

AI/ML is being used to optimize every stage of the multimedia delivery pipeline.

Content-Aware Encoding: An AI model analyzes the video content frame-by-frame. It allocates more bits to complex, high-motion scenes and fewer bits to simple, static scenes, achieving a much better quality-to-bitrate ratio than a standard fixed-bitrate encoder.

Predictive Rate Adaptation: Instead of just reacting to past bandwidth, a DASH client can use an ML model to **predict** the likely future network throughput based on factors like the radio technology, time of day, and mobility. This allows it to make much smarter and more proactive quality-switching decisions, improving QoE.

Multimedia Networking: Key Takeaways

The Goal: The ultimate objective is to maximize the user's **Quality of Experience (QoE)**, not just the network's Quality of Service (QoS).

Real-time Media: Is powered by **UDP/RTP** for low-latency transport, **RTCP** for QoS feedback, and **SIP** for session signaling. The key mechanism is the **playout buffer** to handle jitter.

On-Demand Media: Is dominated by **HTTP-based Adaptive Streaming (HAS)**, with **MPEG-DASH** as the standard. The key is the **client-side rate adaptation algorithm** that intelligently selects quality levels.

The System: High-quality global streaming is a systems achievement, relying on the perfect synergy between efficient codecs (like H.265/AV1), intelligent client protocols (DASH), and massive distributed systems (**CDNs**).

The Future: Is driven by the need for lower latency (LL-DASH), more immersive experiences (VR), and smarter optimization (AI/ML), with **MEC** as a key enabler.



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

03.3: Multimedia Networking

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

daniele.tarchi@unifi.it